

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ
MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN ĐIỆN TỬ**



BÁO CÁO ĐỒ ÁN THỰC TẬP KỸ SƯ

**ĐỀ TÀI: HIỆN THỰC KỸ THUẬT INSTRUCTION
SET RANDOMIZATION CHO KIẾN TRÚC
RISC-V 32 BIT**

GVHD: TS. TRƯỜNG QUANG VINH

HỌ VÀ TÊN	MÃ SỐ SINH VIÊN
NGUYỄN NHẬT QUANG	2212746

TP. HỒ CHÍ MINH

Mục lục

I. GIỚI THIỆU CHUNG	4
1. Bối cảnh và động lực	4
2. Nguyên lý Kỹ thuật Instruction-Set Randomization (ISR)	5
3. Lựa chọn kiến trúc RISC-V	6
4. Mục tiêu và phạm vi đề tài	6
4.1. Về mục tiêu	6
4.2. Về phạm vi	6
II. CƠ SỞ LÝ THUYẾT	7
1. Tổng quan hệ thống bảo mật với ISR	7
2. Cơ sở lý thuyết về Pipeline RISC-V và cơ chế xử lý Hazard	7
2.1. Rủi ro dữ liệu (Data Hazard) và cơ chế chuyển tiếp Forwarding	7
2.2. Giới hạn Load-use và khối phát hiện rủi ro Hazard Detection Unit	9
2.3. Sự đồng bộ Tín hiệu qua Thanh ghi Pipeline (ID/EX Register)	10
2.4. Kết luận dẫn dắt	12
3. Kiến trúc lõi vi xử lý Ibex RISC-V	12
3.1. Giới thiệu tổng quan về Ibex	12
3.2. Tổ chức pipeline của lõi Ibex	13
3.3. Giao tiếp bộ nhớ lệnh và khối Prefetch Buffer	13
3.4. Đi sâu vào tầng Instruction Fetch – Vị trí can thiệp ISR	14
4. Các thuật toán mật mã áp dụng cho ISR	15
4.1. Yêu cầu về độ trễ phần cứng	15
4.2. Phân tích lựa chọn thuật toán	15
5. Nền tảng FPGA Xilinx Kria KV260	16
5.1. Tổng quan kiến trúc Zynq UltraScale+ MPSoC	16
5.2. Sự tương tác giữa PS và PL trong hệ thống ISR	16
III. THIẾT KẾ VÀ HIỆN THỰC	17
1. Thiết kế Module giải mã phần cứng	17
2. Hiện thực công cụ phần mềm	18
3. Chuyển đổi giao thức và đóng gói IP	20
3.1. Tích hợp IF stage và viết AXI Wrapper	20
3.2. Đóng gói Custom IP qua 6 bước	21
4. Tích hợp hệ thống	22
5. Quá trình triển khai và kiểm thử trên Kit Kria KV260	25
5.1. Tổng quan về môi trường triển khai	25
5.2. Vai trò của phần mềm Mobaxterm	25
5.3. Vai trò của phần mềm Vivado Hardware Manager	25
5.4. Quy trình vận hành kết hợp	26
IV. TỔNG HỢP THIẾT KẾ VÀ PHÂN TÍCH ĐÁNH GIÁ	26
1. Phân tích chi phí tài nguyên	26
2. Phân tích thời gian và tần số	27
3. Phân tích năng lượng tiêu thụ	29

V. KIỂM CHỨNG HỆ THỐNG	30
1. Nhóm 1: Baseline – Không có ISR	30
2. Nhóm 2: Có bật ISR, mã hóa đúng	32
3. Nhóm 3: Kích bản tấn công	34
VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	36
1. Kết luận	36
2. Hướng phát triển	37
TÀI LIỆU THAM KHẢO	38

Danh sách hình vẽ

1	Nguyên lý Instruction-Set Randomization	6
2	Sơ đồ nguyên lý hệ thống có ISR	7
3	Lưu đồ giải thuật Forwarding Unit	8
4	Dạng sóng mô phỏng sự định tuyến dữ liệu của khối Forwarding	8
5	Lưu đồ logic cơ chế chen Bubble của Hazard Detection Unit	9
6	Dạng sóng mô phỏng Hazard Detection Unit	10
7	Sơ đồ khối chốt dữ liệu Always Block của thanh ghi ID/EX	11
8	Dạng sóng mô phỏng thanh ghi ID/EX	11
9	Kiến trúc của lõi ibex	13
10	Kiến trúc hệ thống Ibex có ISR	15
11	Kiến trúc phần cứng ISR (Sơ đồ khối module giải mã mã lệnh sử dụng phép toán logic XOR)	17
12	Dạng sóng mô phỏng chứng minh khôi phục được lệnh RISC-V gốc	18
13	Kiến trúc phần mềm ISR	19
14	Kết quả file vmem trước và sau khi được mã hóa/giải mã toàn vẹn bởi tool Python	20
15	Lưu đồ máy trạng thái chuyển đổi giao thức từ OBI sang AXI4	21
16	Đóng gói thành công IP chứa lõi Ibex	22
17	Sơ đồ kiến trúc bus hệ thống	22
18	Cấu trúc phân luồng 2 cổng AXI Master (m_axi_instr và m_axi_data) trong khối Ibex	23
19	Block design	24
20	Report Utilization thiết kế có ISR	27
21	Report Utilization thiết kế baseline	27
22	Timing Summary thiết kế có ISR	28
23	Timing Summary thiết kế baseline	28
24	Power Summary thiết kế có ISR	29
25	Power Summary thiết kế baseline	30
26	Quy trình kiểm chứng hệ thống	30
27	Kết quả kiểm thử nhóm 1	31
28	Nạp lên kit KV260 lõi ibex không có ISR	32
29	Kết quả kiểm thử nhóm 2	33
30	Nạp lên kit KV260 lõi ibex có ISR nhóm 2	34
31	Kết quả kiểm thử 2 trường hợp nhóm 3	35
32	Nạp lên kit KV260 lõi ibex có ISR nhóm 3	36

Danh sách bảng

1	So sánh tài nguyên phần cứng sau Implementation	26
2	So sánh các chỉ số Timing	28

I. GIỚI THIỆU CHUNG

1. Bối cảnh và động lực

Trong kỷ nguyên Internet vạn vật (IoT) và hệ thống nhúng thông minh, các thiết bị vi xử lý ngày càng được triển khai rộng rãi trong các cơ sở hạ tầng trọng yếu, từ xe tự hành, thiết bị y tế cho đến các hệ thống điều khiển công nghiệp. Sự bùng nổ này mang lại nhiều tiện ích nhưng đồng thời cũng mở ra một bề mặt tấn công rộng lớn cho giới tội phạm mạng. Các thiết bị nhúng thường bị giới hạn về tài nguyên (diện tích silicon, công suất tiêu thụ, bộ nhớ) nên việc triển khai các cơ chế bảo mật phần mềm truyền thống nặng nề, như các phần mềm diệt virus hay tường lửa phức tạp, thường không khả thi.

Một trong những mối đe dọa dai dẳng và nguy hiểm nhất đối với các hệ thống máy tính nói chung và hệ thống nhúng nói riêng là các cuộc tấn công khai thác lỗ hổng bộ nhớ. Thông qua các lỗi lập trình phổ biến như tràn bộ đệm, kẻ tấn công có thể ghi đè lên bộ nhớ của chương trình đang chạy. Từ đó, chúng có thể thực hiện hai hình thức tấn công nguy hiểm:

1. **Tấn công chèn mã (Code Injection):** Kẻ tấn công chèn trực tiếp các đoạn mã độc vào bộ nhớ dữ liệu và bẻ lái luồng thực thi của chương trình trở tới đoạn mã đó để chiếm quyền điều khiển hệ thống.
2. **Tấn công tái sử dụng mã (Code Reuse Attack - điển hình là ROP - Return-Oriented Programming):** Khi các cơ chế bảo vệ như Data Execution Prevention (DEP) hoặc Non-Executable (NX) bit ngăn chặn việc thực thi mã ở vùng dữ liệu, kẻ tấn công chuyển sang ghép nối các mảnh mã hợp lệ có sẵn trong bộ nhớ để tạo ra một luồng thực thi độc hại mới.

Điểm chung cốt lõi của các phương thức tấn công này là chúng đều đòi hỏi kẻ tấn công phải biết trước và hiểu rõ tập lệnh (Instruction Set Architecture - ISA) mà vi xử lý mục tiêu đang sử dụng. Mã độc được chèn vào hoặc các gadget được tìm kiếm đều phải tuân thủ nghiêm ngặt định dạng mã máy (machine code) của kiến trúc đó. Trên thực tế, các hệ thống phần cứng thường được sản xuất hàng loạt với cùng một cấu hình vi kiến trúc và chạy các tập lệnh chuẩn hóa, biến chúng thành các "mục tiêu tĩnh". Chỉ cần kẻ tấn công dịch ngược và khai thác thành công một thiết bị, chúng có thể dễ dàng nhân bản cuộc tấn công đó lên hàng triệu thiết bị cùng loại.

Từ thực trạng trên, nhu cầu bức thiết đặt ra là phải có một cơ chế bảo vệ chủ động, phá vỡ tính "đồng nhất" của các hệ thống để ngăn chặn các cuộc tấn công dựa trên binary ngay từ cấp độ phần cứng. Đây chính là động lực cốt lõi dẫn đến sự ra đời của kỹ thuật Instruction Set Randomization (ISR).

Bằng cách ngẫu nhiên hóa (mã hóa) tập lệnh riêng biệt cho từng thiết bị (hoặc từng tiến trình), ISR biến đổi định dạng lệnh hợp lệ thành một chuỗi bit ngẫu nhiên đối với thế giới bên ngoài. Khi đó, bất kỳ mã độc nào được chèn vào bởi kẻ tấn công (vốn tuân theo tập lệnh chuẩn) đều sẽ không khớp với cấu hình giải mã bên trong vi xử lý. Kết quả là, vi xử lý sẽ giải mã đoạn mã độc thành các lệnh vô nghĩa hoặc không hợp lệ (Illegal Instruction), gây ra ngoại lệ (exception) và ngay lập tức dừng chương trình trước khi mã độc kịp thực thi.

Trong bối cảnh kiến trúc mã nguồn mở RISC-V đang trỗi dậy mạnh mẽ và dần trở thành tiêu chuẩn mới cho các hệ thống nhúng, việc nghiên cứu và tích hợp các module bảo mật bằng phần cứng (Hardware-based Security) trực tiếp vào đường ống lệnh (pipeline)

của RISC-V là một hướng đi đầy hứa hẹn. Đề tài tập trung vào việc hiện thực kỹ thuật ISR trên lõi RISC-V không chỉ mang ý nghĩa học thuật trong việc minh chứng khả năng phòng thủ chủ động của phần cứng, mà còn có giá trị thực tiễn cao khi đánh giá chi tiết sự đánh đổi về tài nguyên và tốc độ trên nền tảng SoC FPGA hiện đại như Xilinx Kria KV260.

2. Nguyên lý Kỹ thuật Instruction-Set Randomization (ISR)

Instruction-Set Randomization (ISR) hay "Ngẫu nhiên hóa tập lệnh", là một phương pháp bảo mật mạnh mẽ nhằm chống lại các cuộc tấn công tiêm nhiễm mã độc, đặc biệt là lỗi tràn bộ đệm. Kẻ tấn công có thể can thiệp vào các chuỗi nhị phân kích hoạt mã lệnh trên CPU dưới dạng mã máy chuẩn, khiến cho CPU bị đánh lừa thực hiện các lệnh khác. Kỹ thuật ISR giải quyết vấn đề này bằng cách tạo ra 1 ngôn ngữ riêng cho CPU theo cơ chế sau:

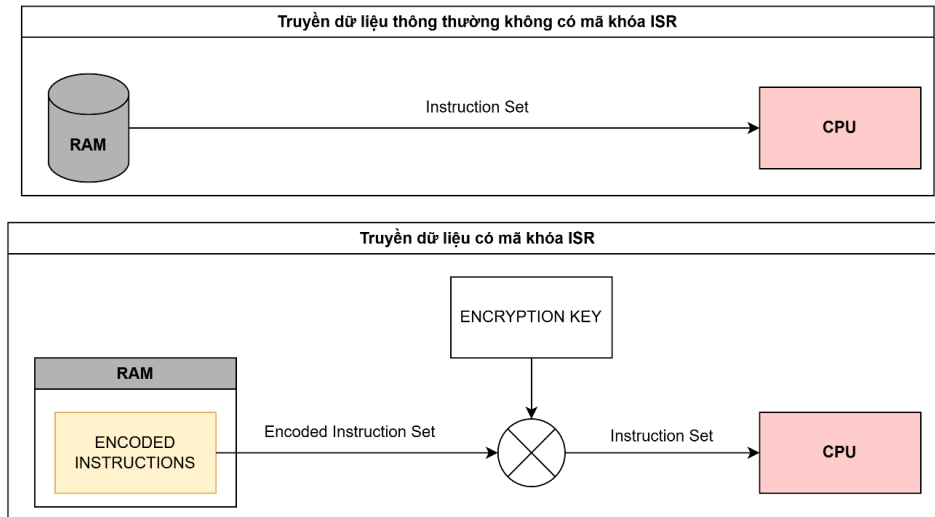
- **Pha Mã hóa (Software-side):** Trước khi chương trình được nạp vào bộ nhớ lệnh (Instruction Memory), toàn bộ file nhị phân chuẩn (.bin) sẽ được đưa qua một công cụ phần mềm (Tool Encoder). Công cụ này sử dụng một Khóa Bí Mật (Secret Key) để xáo trộn toàn bộ mã máy. Cụ thể trong đề án này, thuật toán được sử dụng là bitwise XOR:

$$Mã_gốc \oplus Khóa = Mã_xáo_trộn \quad (1)$$

Nội dung nằm trong RAM lúc này đối với một CPU thông thường chỉ là một ký tự rác vô nghĩa.

- **Pha Giải mã (Hardware-side):** Tại tầng phần cứng lõi Ibex, một module Hardware Decoder sẽ được chèn vào giữa bus đọc dữ liệu từ RAM (`instr_rdata_i`) và khối giải mã (`id_stage`). Mạch phần cứng này lưu trữ cùng một Khóa Bí Mật và áp dụng phép tính ngược lại ngay tại thời gian thực (Real-time). Nhờ đặc tính đối xứng của cổng logic XOR ($Mã_xáo_trộn \oplus Khóa = Mã_gốc$), lệnh rác lại biến thành mã máy hợp lệ trước khi CPU kịp đọc. Việc dùng cổng XOR giúp độ trễ của quá trình giải mã phần cứng là cực kỳ nhỏ, gần như không ảnh hưởng đến Critical Path của CPU.

⇒ **Cơ chế chống Hacker:** Giả sử kẻ tấn công thành công tiêm một đoạn mã độc (là các lệnh RISC-V hợp lệ) vào hệ thống. Vì hacker không biết Khóa Bí Mật, mã độc này không được xáo trộn. Khi đoạn mã độc đi qua Hardware Decoder của CPU, nó sẽ bị mạch XOR biến đổi thành các chuỗi nhị phân vô nghĩa. CPU sẽ ngay lập tức bắt lỗi không nhận diện được lệnh (Illegal Instruction Exception) và treo hệ thống một cách an toàn, chặn đứng hoàn toàn cuộc tấn công.



Hình 1: Nguyên lý Instruction-Set Randomization

3. Lựa chọn kiến trúc RISC-V

Kiến trúc RISC-V được lựa chọn là ibex-core, ban đầu là "Zero-riscy", được phát triển bởi nhóm nghiên cứu tại Đại học ETH Zurich và Đại học Bologna (thuộc dự án PULP platform). Hiện nay, nó được tiếp quản, duy trì và thương mại hóa bởi lowRISC. Điểm đặc biệt khiến Ibex được giới chuyên môn đánh giá cao là việc nó được Google và lowRISC chọn làm "Bộ não cốt lõi" (Root of Trust) cho dự án bảo mật phần cứng OpenTitan. Việc một lõi mã nguồn mở được dùng trong thiết kế chip bảo mật cấp công nghiệp chứng tỏ mã nguồn của Ibex đã được kiểm chứng cực kỳ gắt gao, hoạt động ổn định và có độ tin cậy rất cao khiến đây là một lựa chọn lý tưởng cho dự án này.

4. Mục tiêu và phạm vi đề tài

4.1. Về mục tiêu

- Lý thuyết (nghiên cứu RISC-V, Ibex, ISR).
- Phần cứng (thiết kế module giải mã và can thiệp sâu vào pipeline).
- Phần mềm (toolchain mã hóa).
- Phân tích & kiểm chứng (tổng hợp trên Vivado và chạy thực tế).

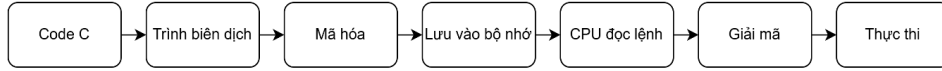
4.2. Về phạm vi

- Chỉ tập trung vào lõi Ibex (không làm các lõi phức tạp hơn).
- Chỉ bảo vệ Instruction Memory (không bảo vệ Data Memory).
- Sử dụng thuật toán mã hóa nhẹ để tối ưu Timing (chỉ XOR với 1 key).
- Sử dụng phần mềm Vivado và board KV260 để đánh giá.

II. CƠ SỞ LÝ THUYẾT

1. Tổng quan hệ thống bảo mật với ISR

Sơ đồ nguyên lý của hệ thống khi có ISR (Mã hóa ở Software Toolchain trước khi nạp, sau đó giải mã trực tiếp bằng Hardware trong quá trình thực thi).



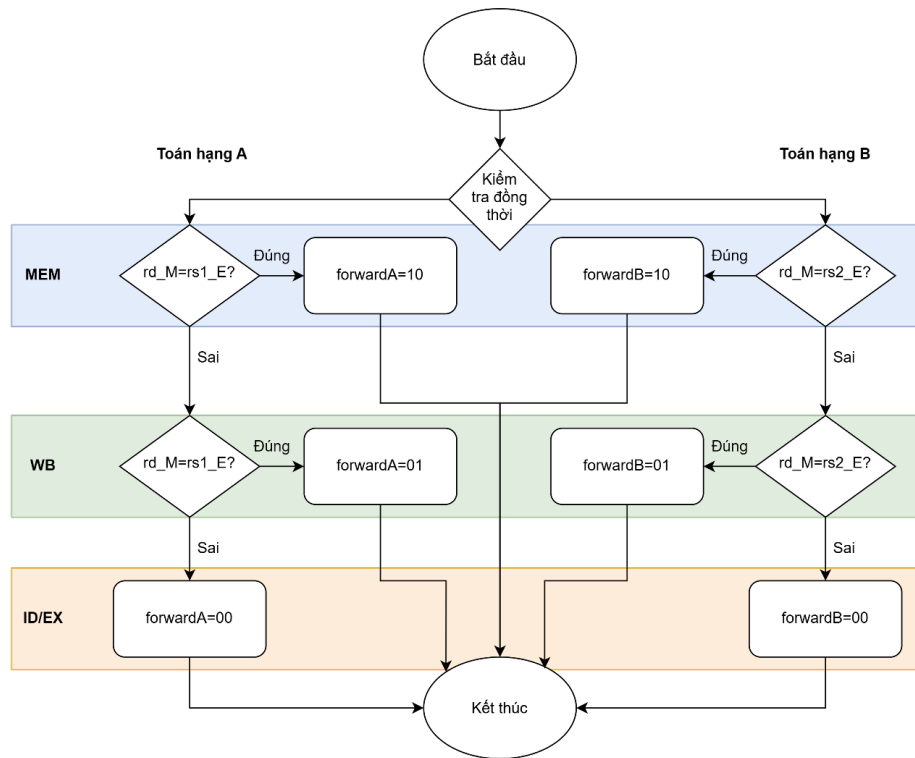
Hình 2: Sơ đồ nguyên lý hệ thống có ISR

2. Cơ sở lý thuyết về Pipeline RISC-V và cơ chế xử lý Hazard

Để có thể can thiệp sâu vào vi kiến trúc nhằm tích hợp module bảo mật Instruction Set Randomization (ISR), việc thấu hiểu cơ chế hoạt động của đường ống lệnh và cách bộ xử lý giải quyết các rủi ro (Hazards) là điều kiện tiên quyết. Tiến hành phân tích và mô phỏng các khối mạch cốt lõi trên kiến trúc RISC-V 5 tầng tiêu chuẩn (IF-ID-EX-MEM-WB) bằng ngôn ngữ mô tả phần cứng và kiểm chứng trên môi trường Ubuntu.

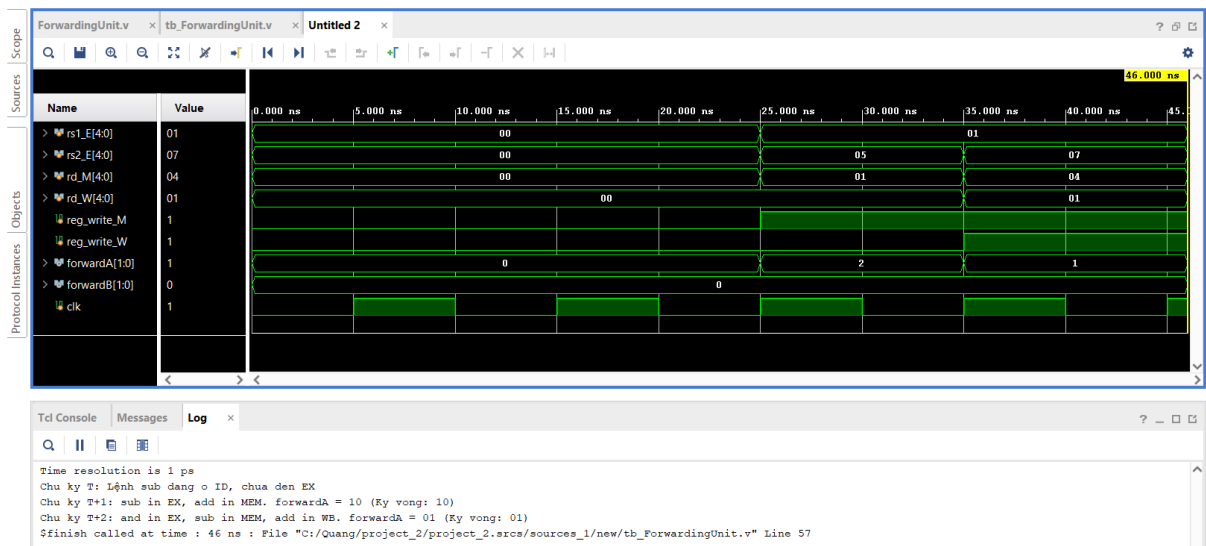
2.1. Rủi ro dữ liệu (Data Hazard) và cơ chế chuyển tiếp Forwarding

Rủi ro dữ liệu xảy ra khi một lệnh phụ thuộc vào kết quả của lệnh đi ngay trước nó, nhưng kết quả này lại chưa được ghi vào tập thanh ghi. Nếu không có cơ chế xử lý, ALU sẽ đọc sai dữ liệu cũ. Giải pháp tối ưu nhất để giải quyết rào cản này là sử dụng khối chuyển tiếp (Forwarding Unit): Thay vì chờ dữ liệu đi hết đường ống, khối này sẽ phát hiện sự phụ thuộc và định tuyến trực tiếp kết quả từ ngõ ra của ALU ở chặng EX/MEM hoặc từ bộ nhớ ở chặng MEM/WB vòng ngược lại ngõ vào của ALU cho lệnh hiện tại.



Hình 3: Lưu đồ giải thuật Forwarding Unit

Lưu đồ logic phát hiện xung đột dữ liệu từ chặng MEM và chặng WB để xuất tín hiệu ForwardA/ForwardB. Để kiểm chứng tính đúng đắn, tiến hành viết Testbench để mô phỏng kịch bản lệnh sub theo ngay sau lệnh add.



Hình 4: Dạng sóng mô phỏng sự định tuyến dữ liệu của khối Forwarding

Phân tích dạng sóng:

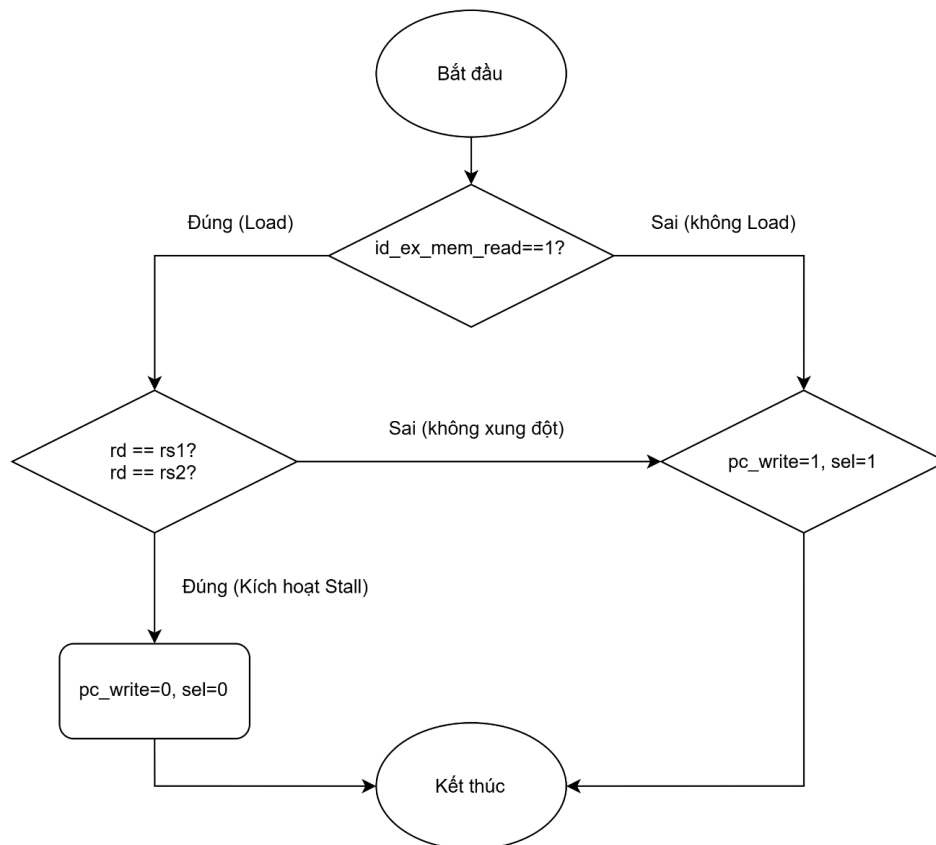
- Giai đoạn Khởi tạo (0ns đến 25ns): Các tín hiệu địa chỉ thanh ghi `rs1_E`, `rs2_E`, `rd_M`, `rd_W` đều bằng 00. Không có xung đột dữ liệu, tín hiệu `forwardA` và `forwardB`

giữ ở mức 0. Đường ống hoạt động bình thường, ALU lấy dữ liệu trực tiếp từ Register File.

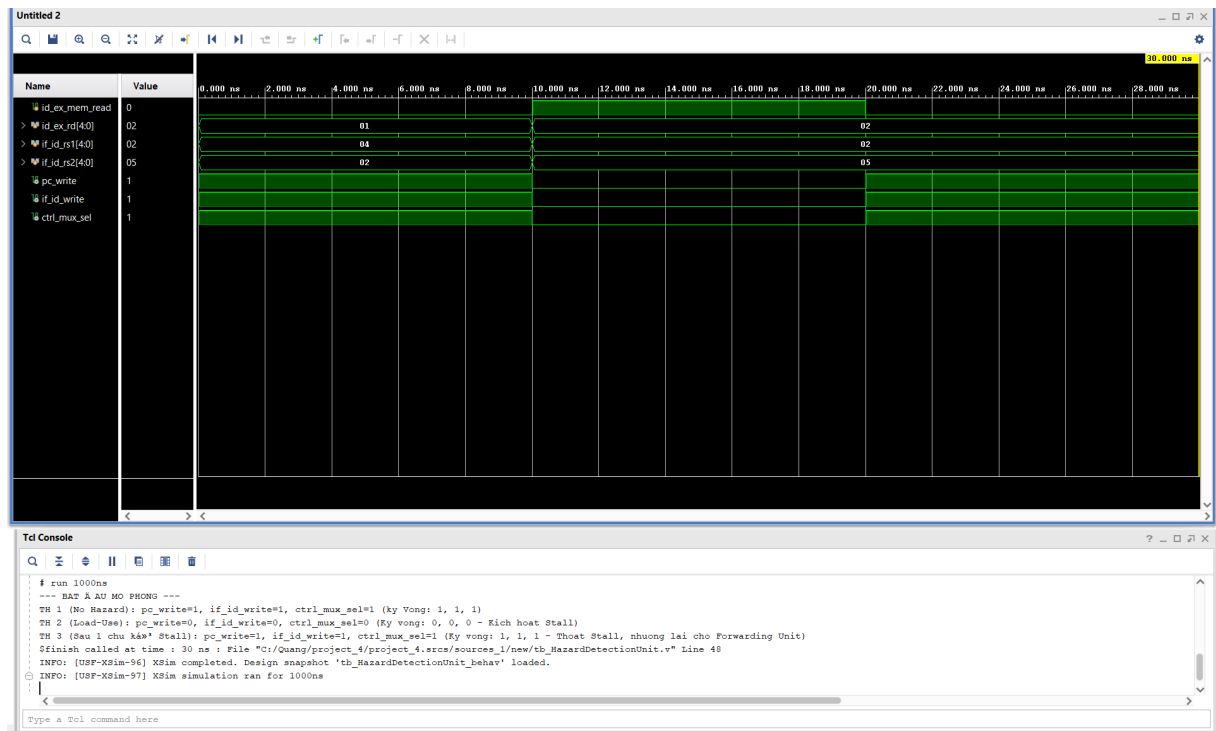
- Chu kỳ T+1 (25ns - 35ns - Giai đoạn MEM): Lệnh add vừa tính xong nằm ở chặng MEM, trong khi lệnh sub cần dữ liệu đang ở chặng EX. Khối Forwarding nhận diện thành công xung đột ($rs1_E = 01$ trùng với $rd_M = 01$). Ngay lập tức, tín hiệu **forwardA** nhảy lên giá trị 2 (10_2), điều khiển Multiplexor bỏ qua việc đọc thanh ghi và lấy thẳng dữ liệu kết quả từ thanh ghi đệm EX/MEM.
- Chu kỳ T+2 (35ns trở đi - Giai đoạn WB): Khi lệnh add trôi xuống chặng WB, tín hiệu **forwardA** chuyển sang giá trị 1 (01_2) để điều khiển Multiplexor kéo dữ liệu từ thanh ghi đệm MEM/WB ngược lại cho ALU. Xuyên suốt quá trình, $rs2_E$ không có xung đột nên **forwardB** vẫn an toàn ở mức 0.

2.2. Giới hạn Load-use và khối phát hiện rủi ro Hazard Detection Unit

Cơ chế Forwarding không thể giải quyết triệt để rủi ro Load-Use (lệnh Load dữ liệu từ bộ nhớ mất nhiều thời gian hơn, trong khi lệnh theo ngay sau lại cần dữ liệu đó ở ngay ALU). Việc dữ liệu phải đi ngược chiều thời gian là bất khả thi về mặt vật lý. Giải pháp duy nhất là phải "đóng băng" (Stall) đường ống. Khối Hazard Detection Unit được thiết kế ở chặng ID để làm nhiệm vụ này.



Hình 5: Lưu đồ logic cơ chế chèn Bubble của Hazard Detection Unit



Hình 6: Dạng sóng mô phỏng Hazard Detection Unit

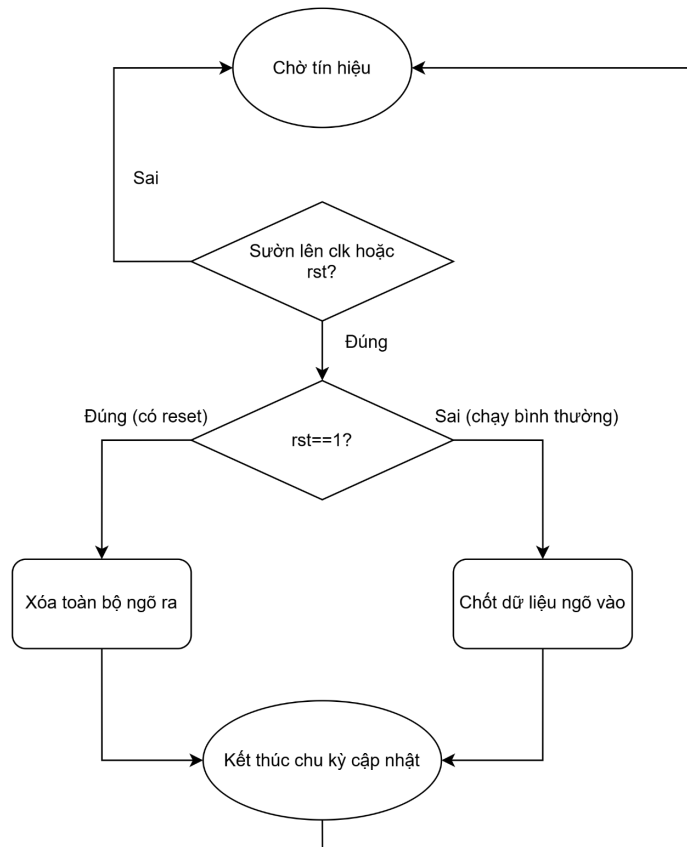
Sự biến đổi của các tín hiệu điều khiển khi phát hiện rủi ro Load-Use.

Phân tích dạng sóng: Khối Hazard là mạch tổ hợp, phản ứng tức thời không cần đợi sườn xung nhịp.

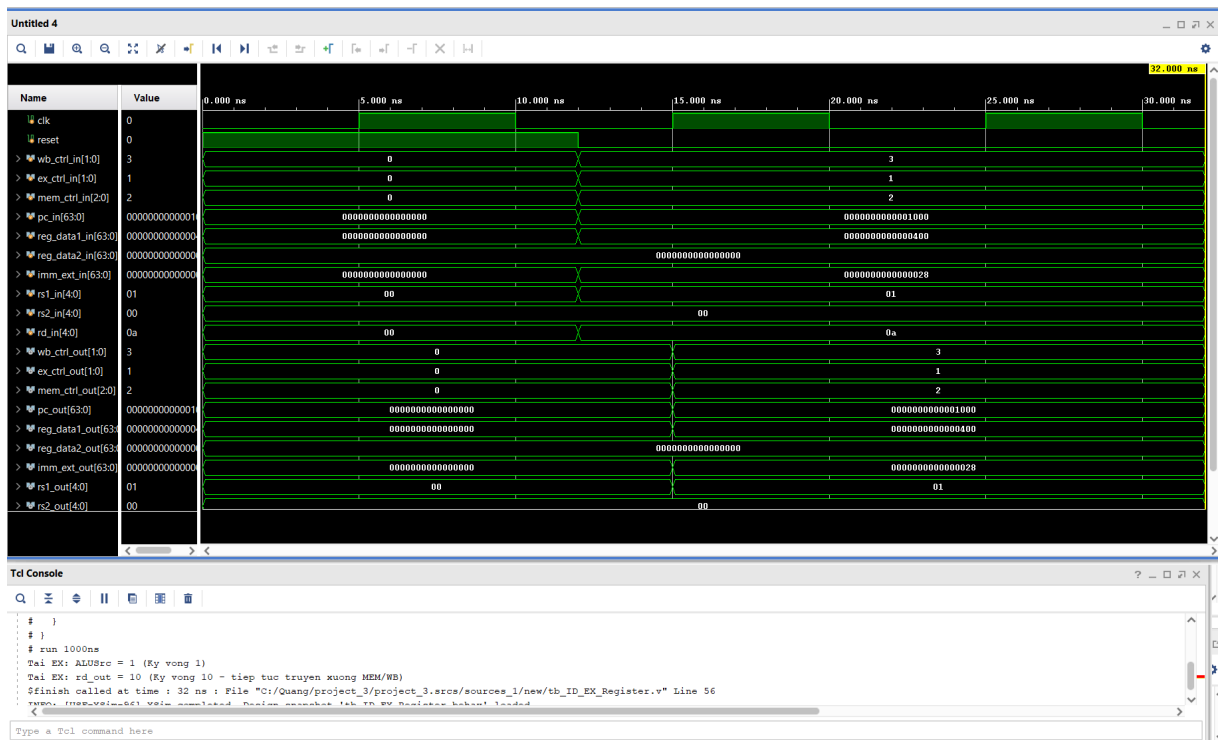
- Từ 0ns - 10ns (Hoạt động bình thường): `id_ex_mem_read` = 0 (lệnh trước không phải Load), mạch nhận diện an toàn. Các tín hiệu điều khiển cốt lõi như `pc_write`, `if_id_write`, `ctrl_mux_sel` đều ở mức cao (1), cho phép PC và bộ đệm lệnh liên tục nạp lệnh mới.
- Từ 10ns - 20ns (Kích hoạt Stall): Tín hiệu `id_ex_mem_read` nhảy lên 1 xác nhận lệnh trước là Load, đồng thời thanh ghi đích của nó (`id_ex_rd` = 02) trùng khớp với thanh ghi nguồn lệnh hiện tại (`if_id_rs1` = 02). Mạch ngay lập tức can thiệp: Kéo rập 3 tín hiệu `pc_write`, `if_id_write` và `ctrl_mux_sel` xuống mức thấp (0). Hậu quả là chặng Fetch bị đóng băng (không tăng PC), chặng ID giữ nguyên lệnh cũ, và một bong bóng lệnh NOP được chèn vào chặng EX.
- Từ 20ns - 30ns (Thoát Stall): Bong bóng NOP đã trôi sang EX (mang theo tín hiệu đọc bộ nhớ bằng 0). Điều kiện Hazard không còn thỏa mãn, các tín hiệu điều khiển lập tức bật lại mức 1. Lệnh kẹt ở ID được giải phóng đi tiếp, lúc này Forwarding Unit đã có thể lấy dữ liệu một cách an toàn.

2.3. Sự đồng bộ Tín hiệu qua Thanh ghi Pipeline (ID/EX Register)

Trong kiến trúc Pipelining, tín hiệu điều khiển không được sinh ra liên tục ở mỗi chặng. Thay vào đó, nó được giải mã một lần ở chặng ID và theo luồng dữ liệu qua các thanh ghi đường ống. Việc thiết kế các thanh ghi này cực kỳ nhạy cảm vì nó là vách ngăn chốt dữ liệu tại mỗi sườn xung nhịp.



Hình 7: Sơ đồ khối chốt dữ liệu Always Block của thanh ghi ID/EX



Hình 8: Dạng sóng mô phỏng thanh ghi ID/EX

Quá trình chốt tín hiệu điều khiển và dữ liệu tại sườn dương của xung nhịp.

Phân tích hoạt động:

- 0ns - 12ns (Giai đoạn Reset): Khi có tín hiệu Reset ở mức cao (1), toàn bộ các ngõ ra của thanh ghi lập tức bị ép về mức 0, làm sạch toàn bộ dữ liệu rác trong đường ống.
- 12ns - 15ns (Bơm dữ liệu ngõ vào): Tín hiệu Reset được hạ. Testbench cấp các giá trị chuẩn bị như `wb_ctrl_in = 3` (Báo hiệu chức năng ghi Register), `imm_ext_in = 0028` (Hằng số bù). Mặc dù ngõ vào đã có dữ liệu, ngõ ra vẫn tuyệt đối giữ mức 0, chặn sự lan truyền dữ liệu không kiểm soát.
- Tại mốc 15ns (Sườn dương Clock): Xung nhịp Clock chuyển từ 0 lên 1. Ngay tại khoảnh khắc sườn lên, bức tường thanh ghi mở ra, lập tức chốt (latch) toàn bộ dữ liệu ngõ vào và đẩy sang ngõ ra. Từ mốc này, chặng EX đã có đầy đủ tín hiệu điều khiển chính xác để tính toán.

2.4. Kết luận dẫn dắt

Việc nghiên cứu, thiết kế và mô phỏng thành công các khối Forwarding Unit, Hazard Detection Unit cùng hệ thống thanh ghi Pipeline trên kiến trúc RISC-V 5 tầng đã giúp nhóm làm chủ hoàn toàn bản chất của việc đồng bộ luồng dữ liệu và điều khiển luồng (Control flow).

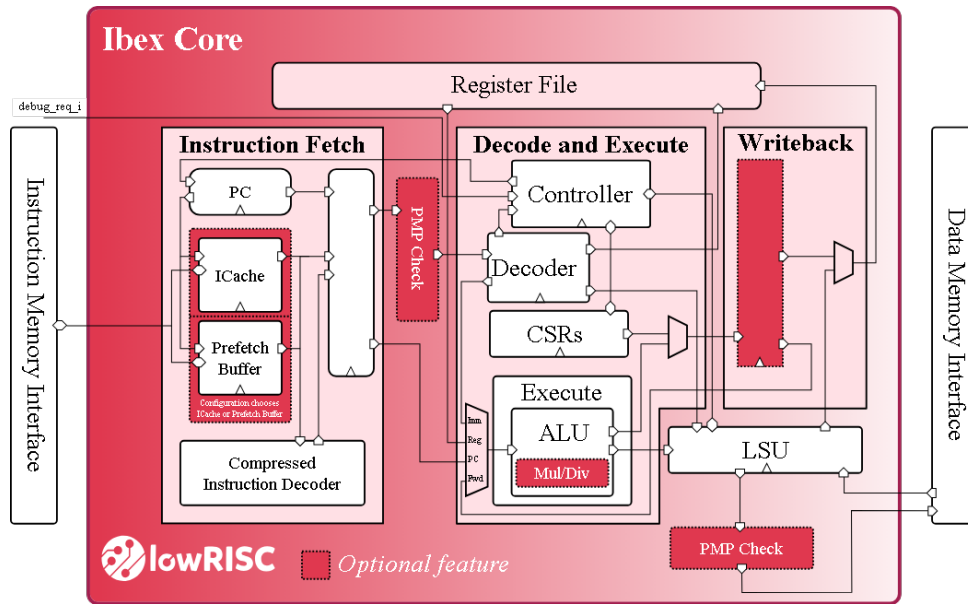
Đây là kiến thức cốt lõi và là tiền đề bắt buộc để giải quyết bài toán lớn của đề tài: Tích hợp module mã hóa ISR. Khi module bảo mật được chèn vào đường truyền lệnh, quá trình giải mã bằng phần cứng chắc chắn sẽ làm phát sinh độ trễ. Để hệ thống không bị crash hoặc đọc sai mã lệnh rác, thiết kế bắt buộc phải áp dụng lại chính các tư duy điều khiển tín hiệu Stall/Flush đã phân tích ở trên, nhằm tương tác hoàn hảo với cơ chế Handshake của CPU, bắt hệ thống Stall luồng Fetch cho đến khi bộ giải mã xuất ra mã lệnh RISC-V hoàn chỉnh.

3. Kiến trúc lõi vi xử lý Ibex RISC-V

3.1. Giới thiệu tổng quan về Ibex

Lõi vi xử lý Ibex (được phát triển và duy trì bởi tổ chức lowRISC) là một bộ vi xử lý 32-bit mã nguồn mở, được thiết kế hoàn toàn tương thích với kiến trúc tập lệnh RISC-V (RV32I/EMC). Đặc điểm nổi bật nhất của Ibex là thiết kế cực kỳ tối ưu về mặt diện tích silicon và tiêu thụ năng lượng, khiến nó trở thành một trong những lõi vi xử lý lý tưởng nhất cho các ứng dụng hệ thống nhúng và Internet vạn vật (IoT).

Thay vì sử dụng các cấu trúc siêu hướng (superscalar) hay thực thi không theo thứ tự (out-of-order execution) phức tạp, Ibex vận hành theo mô hình thực thi tuần tự (in-order execution). Sự đơn giản, gọn nhẹ này hoàn toàn khớp với bối cảnh của đề tài: xây dựng cơ chế bảo mật mà vẫn tối ưu về tài nguyên và hiệu quả.



Hình 9: Kiến trúc của lõi ibex

3.2. Tổ chức pipeline của lõi Ibex

Để duy trì kích thước nhỏ gọn nhất có thể, Ibex được thiết kế với kiến trúc pipeline 2 tầng cốt lõi:

1. **Tầng IF (Instruction Fetch - Nạp lệnh):** Đảm nhiệm việc giao tiếp với bộ nhớ chương trình, lấy mã lệnh (instruction) từ bộ nhớ, xử lý sơ bộ và đưa vào bộ đệm (Prefetch Buffer) để chuẩn bị cho chu kỳ tiếp theo.
2. **Tầng ID/EX (Instruction Decode & Execute - Giải mã và Thực thi):** Nhận lệnh từ tầng Fetch, tiến hành giải mã opcode, đọc dữ liệu từ tập thanh ghi, thực hiện các phép toán logic/số học tại khối ALU, tính toán địa chỉ nhảy, thao tác với bộ nhớ dữ liệu và cuối cùng ghi kết quả ngược lại thanh ghi.

Kiến trúc 2 tầng này giúp Ibex phản ứng cực nhanh với các ngắt và lệnh rẽ nhánh với độ trễ tối thiểu. Tuy nhiên, kiến trúc này cũng đặt ra thách thức về mặt timing nếu ta can thiệp thêm logic giải mã phức tạp vào đường truyền dữ liệu.

3.3. Giao tiếp bộ nhớ lệnh và khối Prefetch Buffer

Để nạp lệnh liên tục và giữ cho pipeline không bị gián đoạn, Ibex không kết nối trực tiếp tầng giải mã với bộ nhớ mà thông qua một module trung gian gọi là Prefetch Buffer (Bộ đệm nạp trước).

Giao tiếp giữa lõi Ibex và bộ nhớ lệnh tuân theo một giao thức handshake tiêu chuẩn bao gồm các tín hiệu cơ bản:

- **instr_req:** Tín hiệu (Master → Slave) yêu cầu cấp quyền đọc lệnh từ bộ nhớ.
- **instr_gnt:** Tín hiệu (Slave → Master) phản hồi xác nhận bộ nhớ đã nhận yêu cầu.
- **instr_rvalid:** Tín hiệu báo rằng dữ liệu trên bus lúc này đã hợp lệ (sẵn sàng để đọc).

- **instr_rdata:** Bus dữ liệu 32-bit chứa mã máy (machine code) của lệnh vừa được nạp.

Prefetch Buffer hoạt động như một hàng đợi nhỏ, liên tục phát tín hiệu **instr_req** để kéo các lệnh tiếp theo từ bộ nhớ thông qua **instr_rdata** ngay cả khi tầng ID/EX đang bận thực thi lệnh trước đó (ví dụ như lệnh nhân/chia tốn nhiều chu kỳ). Sự tồn tại của khối buffer này giúp tối đa hóa băng thông bộ nhớ và che lấp đi độ trễ truy xuất.

3.4. Đi sâu vào tầng Instruction Fetch – Vị trí can thiệp ISR

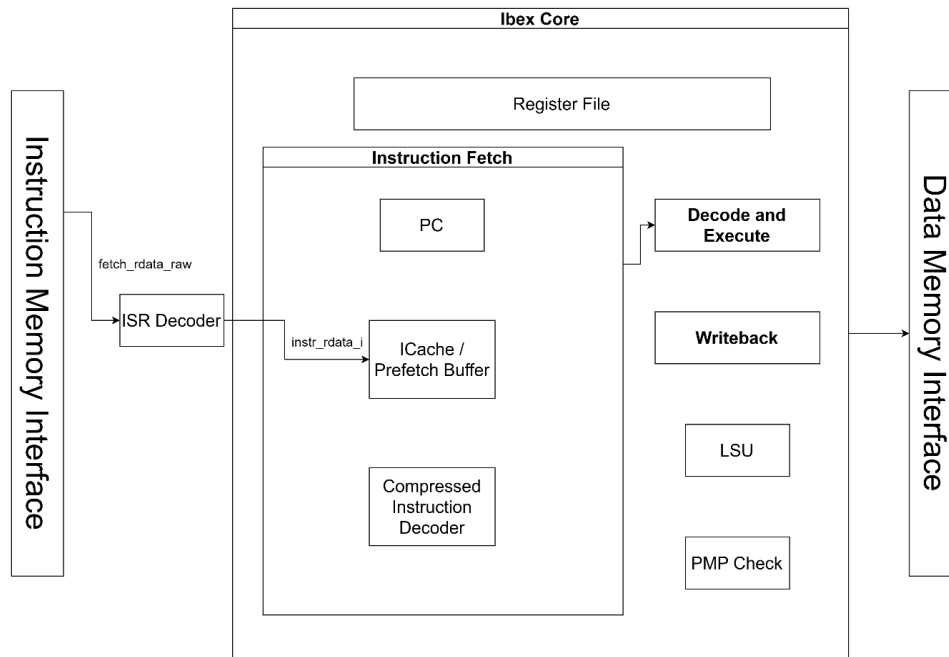
Điểm mấu chốt của kỹ thuật Instruction Set Randomization trong dự án này là việc can thiệp trực tiếp vào dữ liệu mã lệnh trước khi hệ thống kịp thực thi nó. Trong kiến trúc Ibex, khu vực phù hợp nhất để chèn module giải mã phần cứng chính là ngay tại ranh giới giữa luồng dữ liệu trả về của giao thức cấp lệnh (**instr_rdata**) và ngõ vào của khối Prefetch Buffer thuộc tầng IF.

Cụ thể, thay vì để Prefetch Buffer nối trực tiếp với bus **instr_rdata** của bộ nhớ (nơi chứa dữ liệu đã bị mã hóa), đường dữ liệu này sẽ bị "cắt đứt". Module giải mã phần cứng được chèn vào giữa với cơ chế như sau:

- **Ngõ vào của module giải mã:** Nhận dữ liệu mã hóa từ bộ nhớ thông qua bus **instr_rdata**. Đồng thời, module phải bắt được tín hiệu **instr_rvalid** gốc để biết khi nào dữ liệu có sẵn.
- **Ngõ ra của module giải mã:** Trả ra dữ liệu đã giải mã (Plaintext - tức là lệnh RISC-V hợp lệ) và cấp trực tiếp cho ngõ vào của Prefetch Buffer.

Việc can thiệp ở vị trí chiến lược này mang lại hai lợi ích to lớn:

- Nó tách biệt hoàn toàn quá trình giải mã khỏi tầng Decode phức tạp của Ibex. Toàn bộ hệ thống lõi bên trong (Decode, Execute, Hazard Unit) sẽ không hề biết rằng tập lệnh đã từng bị mã hóa, từ đó giữ nguyên vẹn 100% logic xử lý nguyên bản của vi xử lý.
- Quản lý luồng: Thách thức lớn nhất tại vị trí này là điều phối tín hiệu điều khiển. Nếu thuật toán giải mã của ISR tiêu tốn thêm chu kỳ xung nhịp (latency > 0), thiết kế bắt buộc phải giữ lại tín hiệu **instr_rvalid** (không cấp ngay cho Prefetch Buffer) cho đến khi quá trình giải mã hoàn tất 100%, đảm bảo hệ thống không đọc nhầm dữ liệu rác.



Hình 10: Kiến trúc hệ thống Ibex có ISR

4. Các thuật toán mật mã áp dụng cho ISR

4.1. Yêu cầu về độ trễ phân cứng

Khác với việc mã hóa/giải mã tệp tin lưu trữ thông thường (thường được thực hiện bằng phần mềm và không đòi hỏi quá khắt khe về thời gian thực), hệ thống ISR can thiệp trực tiếp vào đường ống lệnh của vi xử lý ở tốc độ rất cao. Cứ mỗi chu kỳ xung nhịp, CPU lại yêu cầu một lệnh mới.

Module giải mã ISR nằm chặn ngay trên đường dẫn tới hạn của chặng Instruction Fetch. Nếu thuật toán giải mã quá phức tạp và tốn nhiều chu kỳ xung nhịp hệ thống sẽ liên tục phải chèn Stall để chờ, làm suy giảm nghiêm trọng chỉ số CPI (Cycles Per Instruction) và thông lượng của CPU. Hơn nữa, logic tổ hợp của bộ giải mã càng dài thì tần số hoạt động tối đa (F_{max}) của toàn bộ lõi RISC-V càng bị kéo thấp xuống. Do đó, yêu cầu tiên quyết để chọn thuật toán ISR là độ trễ siêu thấp, lý tưởng nhất là giải mã xong trong nội bộ một chu kỳ xung nhịp hoặc tốn rất ít tài nguyên phần cứng.

4.2. Phân tích lựa chọn thuật toán

Dựa trên yêu cầu khắt khe về mặt thời gian, các hệ mật mã tiêu chuẩn nặng nề như RSA hay thậm chí là AES chuẩn (tiêu tốn hàng chục cycle) đều bị loại bỏ. Có hai khuynh hướng chính phù hợp cho phần cứng nhúng:

1. Mật mã khối nhẹ:

- Các đại diện tiêu biểu: PRINCE, PRESENT, SIMON, SPECK.
- Ưu điểm: Độ bảo mật cao, được cộng đồng mật mã học thiết kế chuyên biệt cho hệ thống IoT/RFID. Đặc biệt, thuật toán PRINCE được mệnh danh là "low-latency cipher", có khả năng được triển khai dưới dạng tổ hợp hoàn toàn để giải mã ngay lập tức trong 1 chu kỳ clock.

- Nhược điểm: Dù gọi là "nhẹ", nhưng khi triển khai song song toàn bộ các vòng mã hóa, nó vẫn ngốn một lượng lớn LUTs trên FPGA, làm tăng diện tích và công suất tiêu thụ đáng kể so với lõi RISC-V nguyên bản.

2. Mật mã luồng / Mã hóa XOR đơn giản:

- Cơ chế: Sử dụng phép toán logic XOR giữa tập lệnh và một Khóa (Key) tĩnh, hoặc khóa động được sinh ra từ một Bộ tạo số giả ngẫu nhiên (PRNG - Pseudo-Random Number Generator).
- Ưu điểm: Cực kỳ nhanh (độ trễ gần như bằng 0 vì chỉ đi qua 1 cổng logic XOR duy nhất). Tài nguyên phần cứng tiêu tốn không đáng kể, hầu như không ảnh hưởng đến tần số F_{max} của lõi Ibex. Phù hợp tuyệt đối với thiết bị cực kỳ hạn chế tài nguyên.
- Nhược điểm: Độ bền vững mật mã học thấp hơn so với Block Cipher nếu chỉ dùng một khóa tĩnh.

Trong khuôn khổ dự án này, lựa chọn mã hóa XOR với 1 Key tĩnh sẽ dễ dàng được tối ưu và kiểm thử.

5. Nền tảng FPGA Xilinx Kria KV260

5.1. Tổng quan kiến trúc Zynq UltraScale+ MPSoC

Để triển khai, kiểm chứng thực tế cũng như đo lường chính xác các thông số phần cứng, dự án sử dụng kit phát triển Xilinx Kria KV260 Vision AI Starter Kit. Trái tim của bo mạch này là kiến trúc Zynq UltraScale+ MPSoC do hãng Xilinx phát triển.

Khác với các dòng FPGA thuần túy, kiến trúc Zynq MPSoC là một nền tảng không đồng nhất tích hợp cả hai sức mạnh trên cùng một đế silicon:

- **Processing System (PS):** Hệ thống vi xử lý nhân ARM, cung cấp sức mạnh tính toán, có khả năng chạy hệ điều hành hoàn chỉnh như Linux để quản lý mạng, file system và giao diện người dùng.
- **Programmable Logic (PL):** Gồm các ma trận cổng logic khả trình cung cấp tài nguyên phần cứng (LUT, FF, BRAM, DSP) khổng lồ, cho phép triển khai các bộ tăng tốc phần cứng, hay trong trường hợp của dự án này, là nơi ứng dụng lõi vi xử lý RISC-V Ibex và module bảo mật ISR.

5.2. Sự tương tác giữa PS và PL trong hệ thống ISR

Sự tồn tại song song của PS và PL mang lại một mô hình thử nghiệm và quản lý bảo mật cực kỳ ưu việt cho hệ thống.

Trong kiến trúc của đề tài, lõi Ibex mang hệ thống ISR đóng vai trò là một khối phần cứng hoạt động hoàn toàn trong vùng PL. Hệ thống PS đóng vai trò là vi xử lý trung tâm (Master) quản lý toàn bộ quá trình nạp và khởi động lõi RISC-V. Sự giao tiếp giữa hai thế giới này được thực hiện qua hệ thống bus AXI tốc độ cao:

1. Hệ thống PS sử dụng đường truyền băng thông lớn để đẩy chương trình ứng dụng (đã được mã hóa phần mềm) vào không gian bộ nhớ (BRAM/DDR) để lõi RISC-V (PL) chuẩn bị thực thi.

- Quan trọng nhất, hệ thống PS đại diện cho lớp quản lý an toàn. Thông qua bus giao tiếp cấu hình (AXI-Lite), PS sẽ nạp Khóa giải mã mật vào các thanh ghi của khối Key Manager nằm bên trong module ISR ở PL.

Sự phân tách này đảm bảo rằng: Khóa mã hóa nằm an toàn trên phần cứng (PL), không thể bị đánh cắp bởi các phần mềm độc hại chạy trên lõi RISC-V, đồng thời cơ chế cập nhật khóa vẫn diễn ra linh hoạt thông qua sự điều khiển của hệ thống PS.

III. THIẾT KẾ VÀ HIỆN THỰC

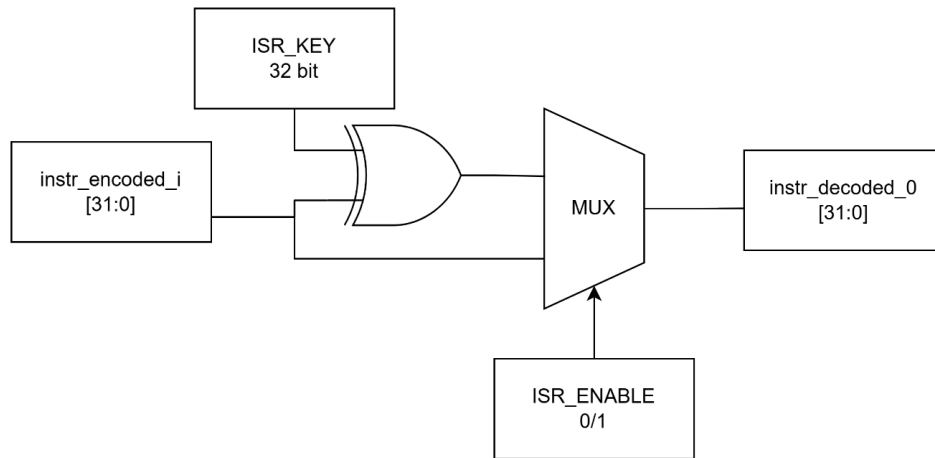
Dựa trên cơ sở lý thuyết đã nghiên cứu ở Chương II, chương này trình bày chi tiết quá trình hiện thực kỹ thuật Instruction Set Randomization lên lõi vi xử lý Ibex. Quá trình được triển khai từ các cấu kiện nhỏ nhất (module giải mã, quản lý khóa) cho đến việc can thiệp vào vi kiến trúc (pipeline), xây dựng công cụ phần mềm và cuối cùng là tích hợp toàn bộ lên hệ thống SoC (System-on-Chip) trên phần mềm Vivado.

1. Thiết kế Module giải mã phần cứng

Trái tim của hệ thống phần cứng là module giải mã (`ibex_isr_decoder.sv`). Để đáp ứng yêu cầu khắt khe về độ trễ như đã phân tích ở mục 2.4 nhằm giữ nguyên vẹn F_{max} của lõi Ibex, dự án sử dụng thuật toán mã hóa XOR kết hợp với một Khóa bảo mật tĩnh độ dài 32-bit (32'hA1B2C3D4).

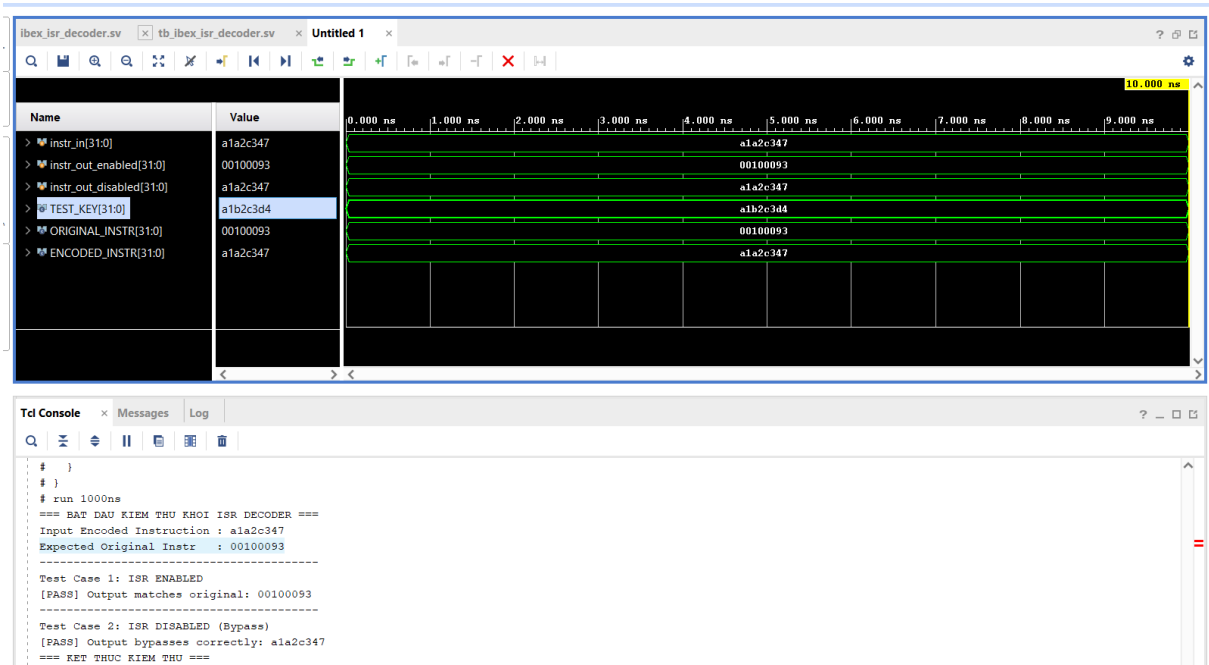
Module được thiết kế với các cổng giao tiếp cốt lõi:

- `ISR_ENABLE`: Cờ tín hiệu điều khiển (1-bit) cho phép bật/tắt tính năng bảo mật.
- `ISR_KEY`: Khóa mã hóa 32-bit.
- `instr_encoded_i`: Luồng mã máy đã bị mã hóa nạp từ bộ nhớ.
- `instr_decoded_o`: Luồng mã lệnh chuẩn trả về cho đường ống lệnh.



Hình 11: Kiến trúc phần cứng ISR (Sơ đồ khối module giải mã mã lệnh sử dụng phép toán logic XOR)

Nguyên lý hoạt động: Mạch hoàn toàn là tổ hợp. Nếu $ISR_ENABLE = 1$, tín hiệu đầu ra $instr_decoded_o = instr_encoded_i \oplus ISR_KEY$. Việc sử dụng phép XOR đảm bảo quá trình giải mã tiêu tốn 0 chu kỳ xung nhịp. Nếu mạch bị tắt, dữ liệu đi thẳng (Bypass). Bất kỳ dữ liệu rác nào sinh ra do sai khóa sẽ lập tức khiến khối Decode của RISC-V kích hoạt cờ Exception (Trap).



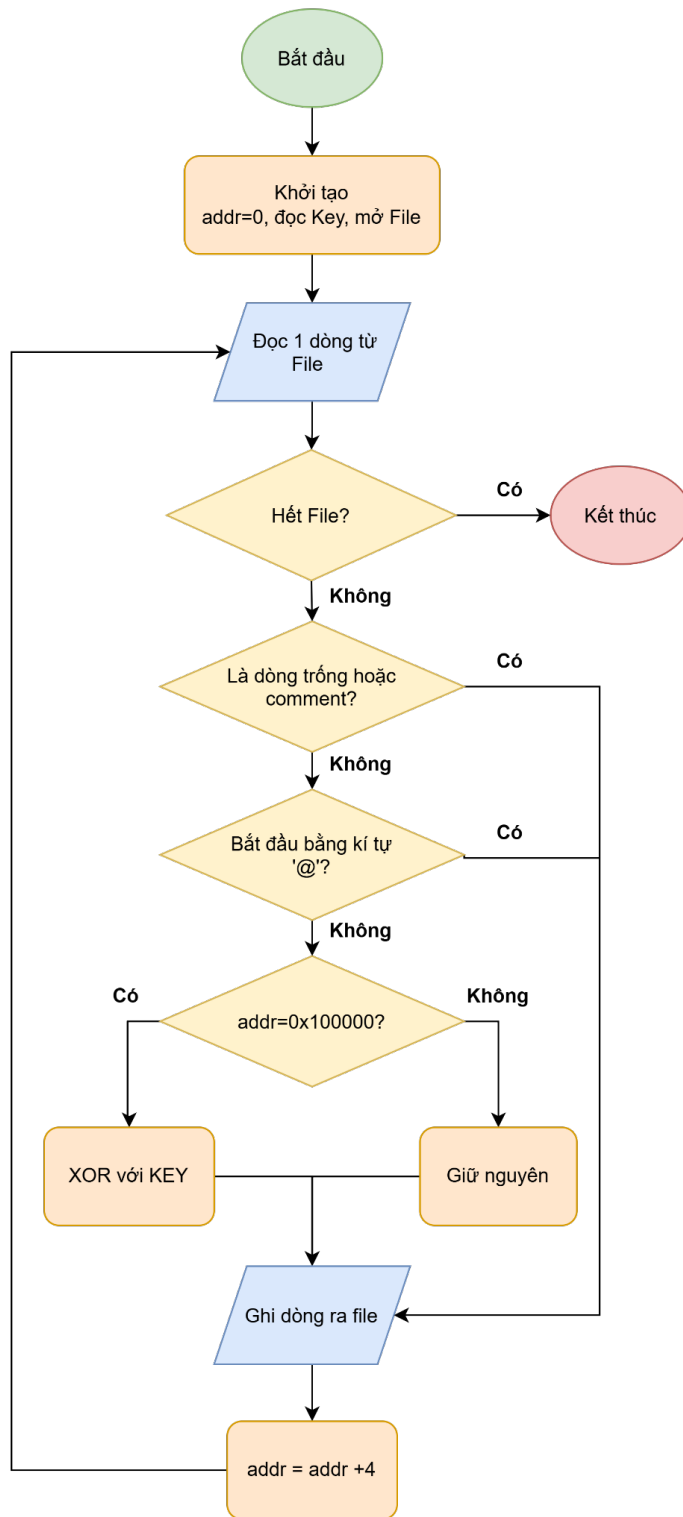
Hình 12: Dạng sóng mô phỏng chứng minh khôi phục được lệnh RISC-V gốc

2. Hiện thực công cụ phần mềm

Phần cứng chỉ có thể giải mã nếu phần mềm đã được mã hóa vì vậy, cần thiết có một công cụ phần mềm để mã hóa file bộ nhớ (.vmem, .hex, .bin) sau khi biên dịch.

Công cụ được lập trình logic để phân tách ranh giới vùng nhớ:

- Sử dụng biến `INSTRUCTION_REGION_END = 0x100000` để làm mốc.
- Phần mềm đọc từng dòng file gốc, chỉ thực hiện phép toán `encoded_instr = original_instr \oplus key` đối với các dòng có địa chỉ nhỏ hơn mốc này (thuộc vùng Instruction/Text). Các dữ liệu thuộc vùng .data hay .rodata được giữ nguyên vẹn để không làm hỏng tính toán của chương trình.
- Tool cũng hỗ trợ cờ `-d` để thực hiện giải mã ngược (Decode) nhằm mục đích kiểm chứng.



Hình 13: Kiến trúc phần mềm ISR

Kiểm thử test_input.vmem với 3 lệnh RV32I cơ bản (addi x1, x0, 1, addi x2, x0, 2, add x3, x1, x2). Chạy lệnh script với khóa ngẫu nhiên (A1B2C3D4):

The image consists of three vertically stacked terminal window screenshots from a virtual machine named 'sunlight'.

The first screenshot shows a nano editor editing 'test_input.vmem'. The content is assembly code:

```
GNU nano 6.2 test_input.vmem
@00000000
// addi x1, x0, 1
00100093
// addi x2, x0, 2
00200113
// add x3, x1, x2
002081B3
```

The second screenshot shows the command line where the file is encoded and then decoded:

```
sunlight@sunlight-virtual-machine: ~/ibex
sunlight@sunlight-virtual-machine:~/ibex$ nano isr_encoder.py
sunlight@sunlight-virtual-machine:~/ibex$ python3 isr_encoder.py -i test_input.vmem -o test_encoded.vmem -k A1B2C3D4
START Ma hoa (Encode) file: test_input.vmem
Khoa (Key): 0xA1B2C3D4
Thanh cong xu ly 3 lenh (instructions).
[+] File ket qua tai: test_encoded.vmem

sunlight@sunlight-virtual-machine:~/ibex$ python3 isr_encoder.py -i test_encoded.vmem -o test_decoded.vmem -k A1B2C3D4 -d
START Giai ma (Decode) file: test_encoded.vmem
Khoa (Key): 0xA1B2C3D4
Thanh cong xu ly 3 lenh (instructions).
[+] File ket qua tai: test_decoded.vmem

sunlight@sunlight-virtual-machine:~/ibex$ nano test_encoded.vmem
sunlight@sunlight-virtual-machine:~/ibex$ nano test_decoded.vmem
sunlight@sunlight-virtual-machine:~/ibex$
```

The third screenshot shows the nano editor editing 'test_encoded.vmem'. The content is the encoded assembly code:

```
GNU nano 6.2 test_encoded.vmem
@00000000
// addi x1, x0, 1
A1A2C347
// addi x2, x0, 2
A192C2C7
// add x3, x1, x2
A1924267
```

Hình 14: Kết quả file vmem trước và sau khi được mã hóa/giải mã toàn vẹn bởi tool Python

3. Chuyển đổi giao thức và đóng gói IP

Quá trình đưa module ISR vào lõi Ibex và chuẩn bị tích hợp lên FPGA Xilinx đòi hỏi sự can thiệp từ vi kiến trúc cho đến giao diện Bus.

3.1. Tích hợp IF stage và viết AXI Wrapper

Bên trong lõi, module `ibex_isr_decoder` được gọi trực tiếp tại file `ibex_if_stage.sv`. Đường dữ liệu thô từ RAM (`fetch_rdata_raw`) bị cắt đứt và bắt buộc phải đi qua module ISR trước khi đi vào khối Prefetch của CPU. Tuy nhiên, lõi Ibex nguyên bản giao tiếp bằng chuẩn OBI. Để tương thích với hệ sinh thái Xilinx Vivado, cần phải tự thiết kế một module AXI Wrapper để dịch tín hiệu từ OBI sang chuẩn AMBA AXI4 (tạo ra 2 cổng Master: `m_axi_instr` chỉ đọc lệnh và `m_axi_data` đọc/ghi dữ liệu).

Khối Wrapper này đóng vai trò như một bộ phiên dịch phần cứng, ánh xạ trực tiếp các tín hiệu của Ibex sang AXI:

- **Phía Ibex (OBI):** Các tín hiệu yêu cầu và dữ liệu thô như `req`, `gnt`, `rvalid`, `addr`,

wdata, rdata.

- **Phía AXI4:** Được chuyển đổi tương ứng thành các kênh chuẩn hóa: kênh địa chỉ (ARADDR, AWADDR), kênh dữ liệu (RDATA, WDATA), và các cờ xác nhận tín hiệu hợp lệ (ARVALID, AWVALID, BVALID). Mọi truy xuất bộ nhớ từ Ibex đều phải đi qua bộ chuyển đổi này.

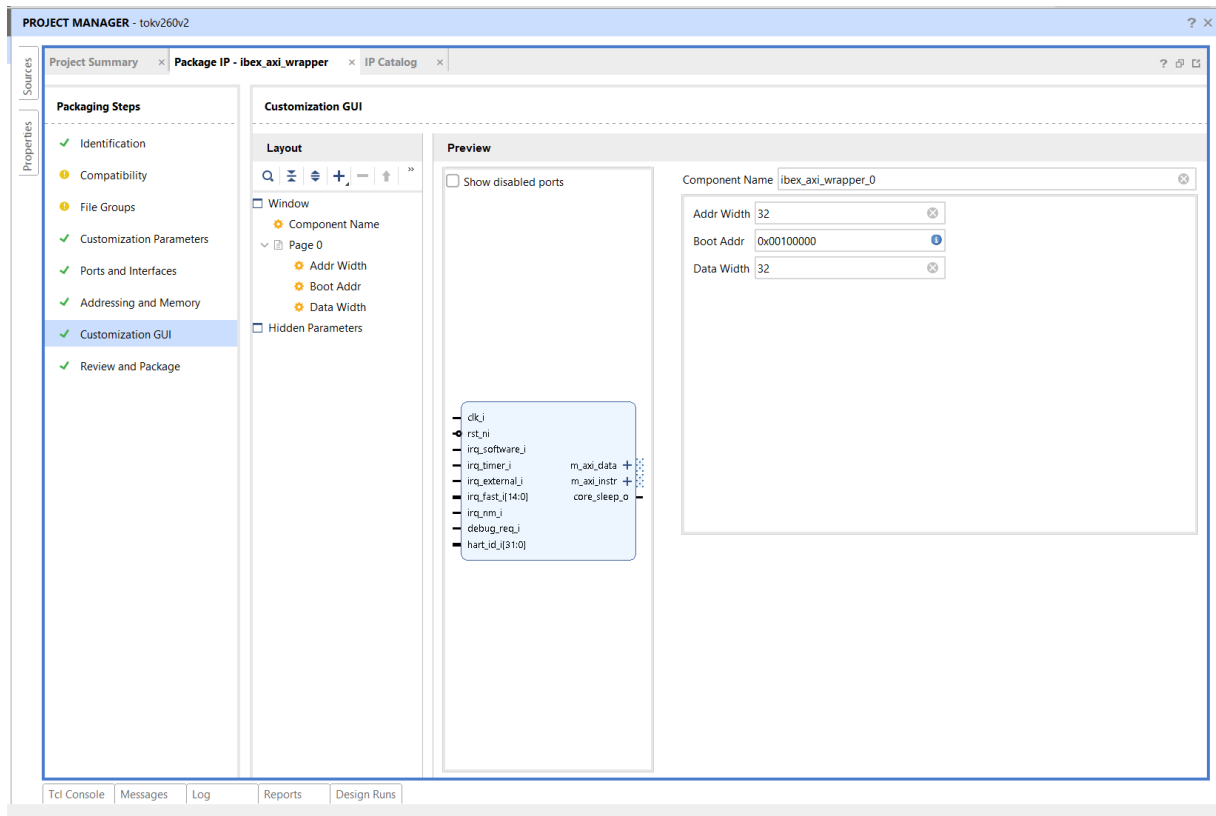


Hình 15: Lưu đồ máy trạng thái chuyển đổi giao thức từ OBI sang AXI4

3.2. Đóng gói Custom IP qua 6 bước

Vì Vivado Block Design không thể tự động ghim nối các tín hiệu rời rạc của RTL, nhóm tiến hành đóng gói toàn bộ lõi Ibex và Wrapper thành 1 Custom IP thông qua quy trình 6 bước khắt khe:

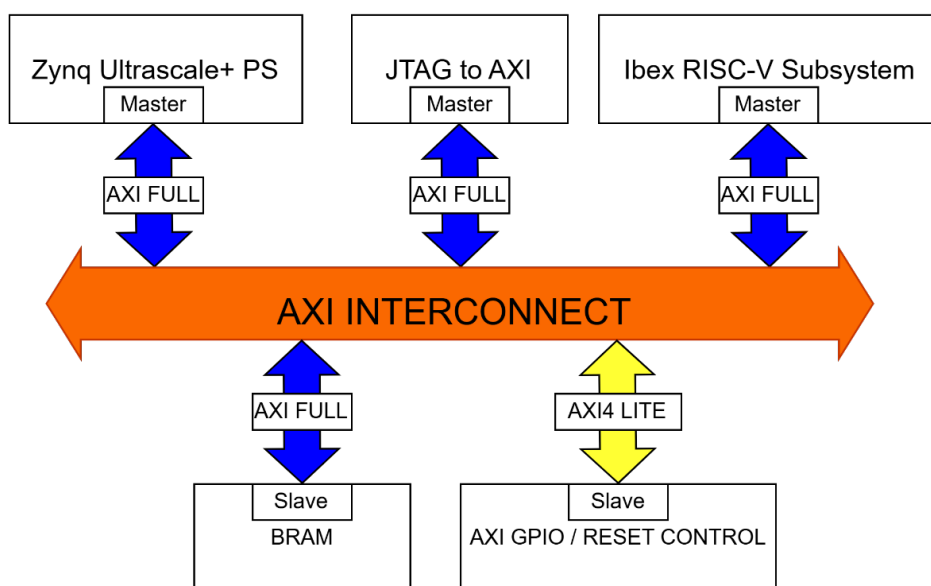
1. **Khởi tạo & Nạp mã nguồn:** Tạo project cấu hình, nạp toàn bộ thư mục RTL của Ibex và set file wrapper làm Top module.
2. **Đơn giản hóa tham số:** Chuyển đổi các tham số SystemVerilog dạng Struct thành dạng Integer cơ bản để Vivado IP Packager có thể nhận diện giao diện đồ họa.
3. **Khởi chạy IP Packager:** Đưa mã nguồn vào không gian làm việc của trình đóng gói.
4. **Cấu hình giao diện Bus:** Tạo thủ công 2 giao diện chuẩn AXI4 (m_axi_instr và m_axi_data), ánh xạ từng chân vật lý vào khe logic. Cài đặt chân clk_i kiểu Clock và rst_ni kiểu Reset (Active Low).
5. **Đồng bộ hóa xung nhịp:** Khai báo thuộc tính ASSOCIATED_BUSIF = m_axi_instr:m_axi_data và đồng bộ tần số FREQ_HZ khớp tuyệt đối với 99.999001 MHz của Zynq PS.
6. **Hoàn tất đóng gói:** Trình xuất ra một thư mục IP hoàn chỉnh sẵn sàng cho SoC.



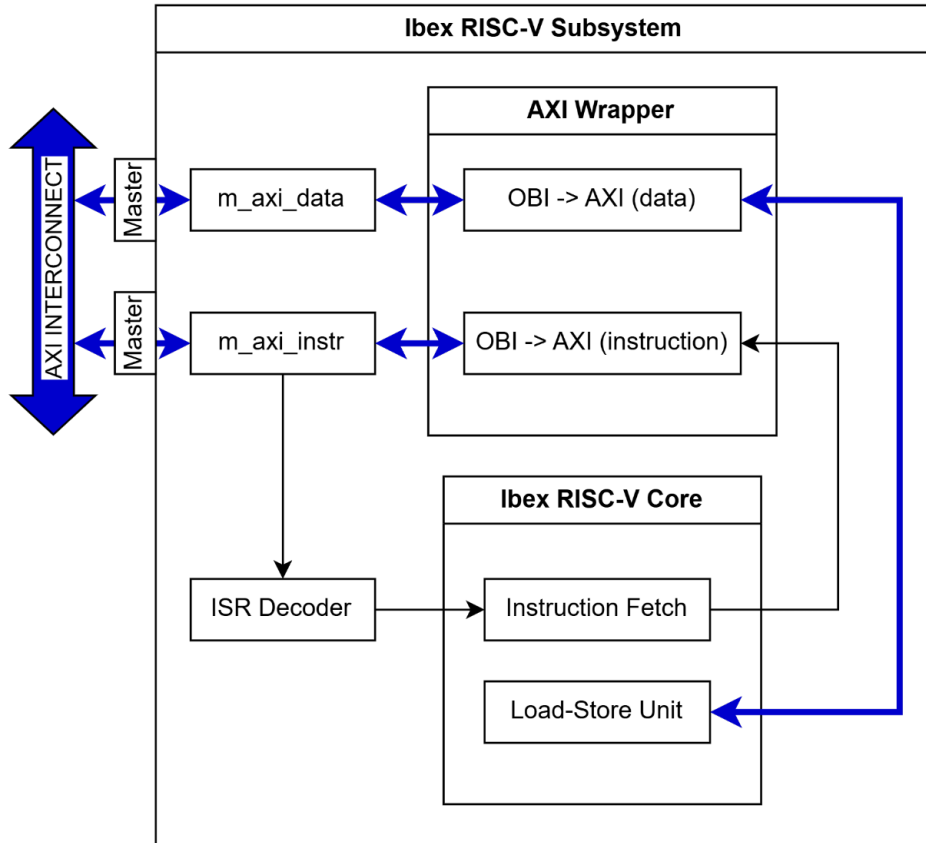
Hình 16: Đóng gói thành công IP chứa lõi Ibex

4. Tích hợp hệ thống

IP Core Ibex (đã có ISR) được lắp ráp thành hệ thống System-on-Chip hoàn chỉnh, mô phỏng kiến trúc đa lõi bất đối xứng: Zynq PS đóng vai trò máy chủ và Ibex đóng vai trò bộ đồng xử lý.

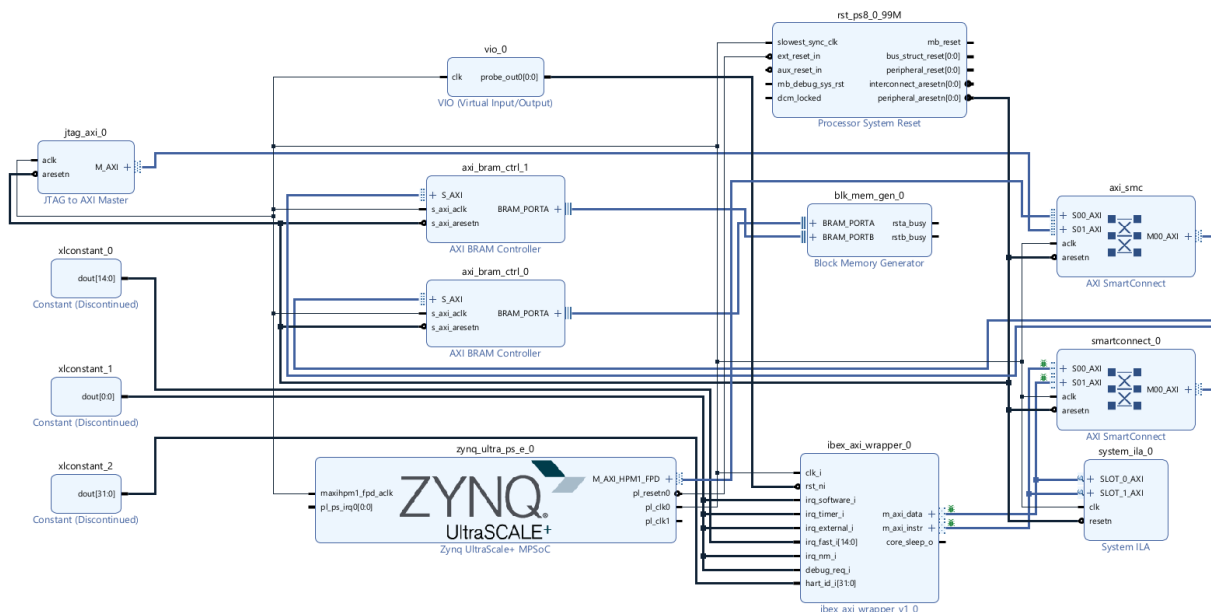


Hình 17: Sơ đồ kiến trúc bus hệ thống



Hình 18: Cấu trúc phân luồng 2 cổng AXI Master (`m_axi_instr` và `m_axi_data`) trong khối Ibex

Như đã thể hiện ở Hình 17, vi xử lý Ibex đóng vai trò là một AXI Master kết nối vào trực tiếp AXI Interconnect. Để làm rõ cấu tạo bên trong của khối Master này cũng như vị trí chuyển đổi giao thức và can thiệp bảo mật, Hình 18 mô tả chi tiết cách thức hai cổng `m_axi_instr` (kênh đọc lệnh có tích hợp giải mã ISR) và `m_axi_data` (kênh truy xuất dữ liệu hai chiều) được ánh xạ từ chuẩn OBI gốc sang chuẩn AXI.



Hệ thống được thiết kế với các khối chức năng tương tác chặt chẽ, đặc biệt chú trọng vào khả năng Hardware Debugging:

1. **Giao tiếp nạp dữ liệu:** Sử dụng hai khối Master là Zynq PS và JTAG to AXI Master. Thông qua trục bus AXI SmartConnect 0, người dùng có thể linh hoạt nạp mã máy đã mã hóa vào BRAM bằng lệnh Python trên Linux hoặc bằng lệnh TCL nạp thẳng qua cáp JTAG.
2. **AXI BRAM:** Đóng vai trò là RAM nội bộ dùng chung (True Dual-port). Cổng A (BRAM_PORTA) nhận dữ liệu nạp từ Host/Debugger. Cổng B (BRAM_PORTB) nối với lõi Ibex để CPU đọc lệnh và ghi kết quả.
3. **Lõi Ibex và AXI SmartConnect:** Lõi Ibex có tới 2 cổng Master (Instruction và Data) cùng cần truy xuất vào cổng B của BRAM. Khối SmartConnect thứ hai làm nhiệm vụ dồn kênh và phân xử, ngăn chặn xung đột khi lệnh đọc và ghi xảy ra cùng lúc.
4. **System ILA:** Được gắn trực tiếp vào hai đường bus `m_axi_instr` và `m_axi_data` của Ibex cho phép bắt và xem dạng sóng thực tế của quá trình nạp lệnh và giải mã ở cấp độ tín hiệu vật lý ngay trên phần mềm Vivado.
5. **Cơ chế Khởi động và Đồng bộ Reset:** Tín hiệu Reset tuyệt đối không được nối trực tiếp từ nguồn điều khiển vào lõi Ibex để tránh rủi ro bất đồng bộ khi nhả reset sử dụng cấu trúc: VIO (Virtual I/O) → Processor System Reset → Ibex `rst_ni`.
 - (a) Khi cấp nguồn, thông qua giao diện VIO trên Vivado, người dùng kích hoạt `probe_out = 1` (Tạo tín hiệu Reset Request). Khối Reset Controller đồng bộ hóa tín hiệu này và ép Ibex ở trạng thái ngủ an toàn.
 - (b) Hệ thống tiến hành bơm file mã máy ISR vào BRAM.
 - (c) Nạp xong, người dùng gạt VIO về mức 0 để nhả Reset. Khối Reset Controller đảm bảo tín hiệu deassert được đồng bộ hoàn toàn với xung nhịp. Lõi Ibex khởi động an toàn, trở PC vào BRAM, tự động giải mã và thực thi.

5. Quá trình triển khai và kiểm thử trên Kit Kria KV260

5.1. Tổng quan về môi trường triển khai

Hệ thống CPU Ibex tích hợp module bảo mật mã hóa tập lệnh (ISR) được triển khai và đánh giá trên nền tảng bo mạch FPGA Xilinx Kria KV260. Do đặc thù kiến trúc Zynq UltraScale+ của KV260 bao gồm hai phần chính là vi xử lý lõi ARM (Processing System - PS) và vùng logic khả trình (Programmable Logic - PL), quá trình triển khai đòi hỏi sự phối hợp chặt chẽ giữa hai công cụ: MobaXterm để quản lý PS và Vivado Hardware Manager để cấu hình, nạp code và gỡ lỗi PL.

5.2. Vai trò của phần mềm Mobaxterm

MobaXterm được sử dụng như một terminal SSH để giao tiếp trực tiếp với hệ điều hành Linux chạy trên nhân ARM (Zynq PS) của bo mạch. Trong đồ án này, MobaXterm đảm nhận vai trò thiết yếu trong khâu khởi tạo nền tảng phần cứng:

- **Khởi tạo Xung nhịp Clock và Cấp nguồn cho PL:** Không giống các FPGA thuần túy, vi xử lý ARM trên KV260 chịu trách nhiệm quản lý xung nhịp và tín hiệu reset cho vùng PL. Thông qua MobaXterm, lệnh `sudo fpgautil -b ibex_wrapper.bit.bin -f Full` được thực thi để nạp bitstream.
- **Giải phóng nút thắt AXI:** Lệnh `fpgautil` từ hệ điều hành Linux sẽ thông báo cho hệ thống quản lý năng lượng (PMU) gỡ bỏ chế độ cách ly giữa PS và PL, đồng thời nhả tín hiệu reset (`pl_reseten0`). Điều này đảm bảo mạng lưới AXI SmartConnect kết nối giữa CPU Ibex và BRAM được cấp điện và hoạt động bình thường, tránh tình trạng kẹt đường truyền khiến tín hiệu ARREADY bị treo ở mức 0.
- **Xóa sạch trạng thái hệ thống:** Việc nạp bitstream qua MobaXterm giúp reset toàn bộ các thanh ghi và Flip-Flop trên PL về trạng thái ban đầu, đảm bảo tính chính xác tuyệt đối trước mỗi lần chạy test case.

5.3. Vai trò của phần mềm Vivado Hardware Manager

Đóng vai trò giám sát quá trình hoạt động của lõi CPU ở mức vật lý và dùng để quan sát dạng sóng. Các công cụ tích hợp trong Vivado được dùng với các mục đích sau:

- **Khối VIO (Virtual Input/Output):** Được sử dụng như một công tắc vật lý ảo để điều khiển trực tiếp chân Reset của lõi Ibex. Thông qua VIO, người kiểm thử có thể chủ động "đóng băng" CPU (đưa Reset về mức 0) để ngăn CPU tự động chạy các lệnh rác khi BRAM chưa có dữ liệu, đảm bảo đường truyền AXI luôn trong trạng thái sẵn sàng.
- **Tcl Console & JTAG-to-AXI:** Thông qua giao diện Tcl của Vivado, các kịch bản phần mềm như `load_bram.tcl` được thực thi. Kịch bản này sử dụng cầu nối JTAG-to-AXI (`hw_axi`) để can thiệp trực tiếp vào BRAM. Nó cho phép mã hóa các lệnh RISC-V thành bản mã bằng thuật toán XOR và ghi trực tiếp vào không gian bộ nhớ của BRAM mà không cần tháo lắp thẻ nhớ hay sử dụng giao tiếp bên ngoài.
- **Khối ILA (Integrated Logic Analyzer):** Đây là công cụ đo kiểm quan trọng nhất để chứng minh tính đúng đắn của thiết kế. ILA được cấu hình để Trigger

ngay tại khoảng khắc CPU nhả Reset và phát tín hiệu xin cấp lệnh ($ARVALID = 1$). ILA chụp lại toàn bộ giao đồ xung quanh các kênh tín hiệu AXI (như ARADDR, RDATA, RRESP...), giúp quan sát trực quan sự chênh lệch giữa lệnh thô và lệnh đã mã hóa, cũng như bắt được chính xác khoảng khắc khối AXI báo lỗi DECERR khi giả lập các cuộc tấn công tiêm mã độc.

5.4. Quy trình vận hành kết hợp

Sự phân công nhiệm vụ rõ ràng giữa MobaXterm và Vivado dẫn đến một quy trình kiểm thử khép kín và an toàn, bao gồm 4 bước tiêu chuẩn:

1. Sử dụng lệnh `fpgautil` trên MobaXterm để cấp xung nhịp và đánh thức vùng PL.
2. Truy cập Vivado VIO để cường chế chân Reset của CPU về 0, giữ hệ thống ở trạng thái tĩnh.
3. Chạy Tcl script trên Vivado để mã hóa và nhồi tập lệnh vào BRAM.
4. Đặt bẫy đo lường trên Vivado ILA, sau đó gạt VIO lên 1 để đánh thức CPU, thu thập giao đồ xung làm minh chứng cho các kịch bản kiểm thử.

IV. TỔNG HỢP THIẾT KẾ VÀ PHÂN TÍCH ĐÁNH GIÁ

Chương này trình bày các kết quả đánh giá phần cứng sau quá trình tổng hợp và triển khai trên phần mềm Xilinx Vivado. Mục tiêu của chương này là chứng minh tính hiệu quả của module ISR: Cung cấp lớp bảo mật phần cứng mạnh mẽ nhưng chỉ tiêu tốn mức tài nguyên cực kỳ thấp và không làm đứt gãy Timing của hệ thống.

Để có cái nhìn khách quan, tiến hành trích xuất báo cáo từ Vivado và so sánh đối chiếu giữa 2 phiên bản lõi CPU được triển khai trên cùng một dòng chip Zynq UltraScale+ của kit Kria KV260:

1. **Baseline:** Lõi Ibex nguyên bản không có tính năng bảo mật.
2. **Ibex-ISR:** Lõi Ibex đã được tích hợp module mã hóa tập lệnh.

1. Phân tích chi phí tài nguyên

Tính tối ưu về diện tích là một trong những chỉ số sống còn khi thiết kế mạch số trên FPGA. Bảng dưới đây so sánh lượng tài nguyên logic bị chiếm dụng giữa hai phiên bản lõi Ibex.

Bảng 1: So sánh tài nguyên phần cứng sau Implementation

Loại tài nguyên	Baseline	Ibex-ISR	Mức độ chênh lệch
LUT	10731	11193	+462 (+4.3%)
FF	10677	13436	+2759 (+25.8%)
BRAM	16	13.5	-2.5
DSP	1	1	0

Nhận xét:

- **Về LUT:** Lượng LUT tăng thêm 462 (chiếm khoảng 4.3%). Sự gia tăng này bao gồm lượng logic tổ hợp (các cổng XOR) của bản thân module giải mã ISR, cộng thêm logic định tuyến của khối System ILA được nhúng vào để phục vụ gỡ lỗi. Mức Overhead 4.3% là mức độ tiêu tốn tài nguyên rất thấp để đổi lấy tính năng bảo mật chèn mã độc.
- **Về FF và BRAM:** Lỗi giải mã ISR thuần tổ hợp nên tiêu tốn 0 FF. Sự gia tăng đột biến về lượng FF (+2,759) là do đã nhúng khối System ILA vào bản Ibex-ISR để phục vụ việc bắt sóng thực tế chứng minh thuật toán hoạt động. Khối ILA này sử dụng lượng lớn FF để làm bộ đệm mẫu, trong điều kiện bình thường, khối ILA này sẽ được gỡ bỏ, đưa mức tiêu thụ FF về tương đương bản gốc. BRAM cũng có sự biến động nhỏ do trình biên dịch tối ưu hóa lại việc cấp phát bộ đệm cho ILA và khối RAM chính.

Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	F8 Muxes (29280)	CLB (14640)	LUT as Logic (117120)	LUT as Memory (57600)	Block RAM Tile (144)	DSPs (1248)	BUFGE (112)	BUFGE_PS (96)	PS8 (1)	BSCANE2 (4)
ibex_wrapper	11193	13437	115	400	2	2467	10114	1079	13.5	1	2	1	1	1
dbg_hub (dbg_hub)	457	765	7	1	0	145	437	20	0	0	1	0	0	1
ibex_j (ibex)	10736	12672	108	399	2	2333	9677	1059	13.5	1	1	1	1	0

Hình 20: Report Utilization thiết kế có ISR

Name	CLB LUTs (117120)	CLB Registers (234240)	CARRY8 (14640)	F7 Muxes (58560)	CLB (14640)	LUT as Logic (117120)	LUT as Memory (57600)	Block RAM Tile (144)	DSPs (1248)	BUFGE (112)	BUFGE_PS (96)	PS8 (1)	BSCANE2 (4)
ibex_wrapper	10731	10678	47	256	2243	9861	870	16	1	2	1	1	1
ibex_j (ibex)	10731	10678	47	256	2243	9861	870	16	1	2	1	1	1

Hình 21: Report Utilization thiết kế baseline

2. Phân tích thời gian và tần số

Module ISR được chèn trực tiếp vào đường dẫn dữ liệu tại tầng Instruction Fetch. Việc chèn thêm cổng logic XOR đồng nghĩa với việc làm tăng thời gian trễ lan truyền. Nếu độ trễ này vượt quá chu kỳ của xung nhịp, hiện tượng vi phạm Timing sẽ xảy ra, làm hệ thống chạy sai hoặc phải giảm tần số.

Tần số hoạt động tối đa được tính toán từ chỉ số Worst Negative Slack (WNS) theo công thức:

$$F_{max} = \frac{1}{T_{clk} - WNS} \quad (2)$$

Trong đó $T_{clk} = 10000$ ns tương ứng với tần số mục tiêu 100 MHz của Zynq UltraScale+. Dựa trên báo cáo Timing Summary, hệ thống ghi nhận giá trị Worst Negative Slack (WNS)

là 0.144 ns trên miền xung nhịp 10.000 ns (100 MHz). Điều này có nghĩa là Fmax tối đa mà thiết kế đạt được sau khi tích hợp ISR là: $F_{max} = 1/(10.000 - 0.144) \text{ ns} \approx 100.001 \text{ MHz}$.

Báo cáo Timing Summary của Vivado cho kết quả như sau tại tần số hoạt động $\sim 100 \text{ MHz}$:

Bảng 2: So sánh các chỉ số Timing

Chỉ số Timing	Baseline	Ibex-ISR	Đánh giá
WNS	0.985 ns	0.144 ns	Pass
TNS	0.000 ns	0.000 ns	Pass
WHS	0.011 ns	0.011 ns	Pass

Nhận xét:

Chỉ số WNS ở bản Ibex-ISR giảm từ 0.985 ns xuống còn 0.144 ns. Sự sụt giảm này là do sự xuất hiện của khối ILA và cổng XOR giải mã trực tiếp trên đường dẫn dữ liệu, làm tăng thời gian trễ lan truyền. Tuy nhiên, WNS vẫn mang giá trị dương (+0.144 ns) và TNS = 0. Điều này chứng minh rằng thiết kế hoàn toàn đạt chuẩn Timing. Module ISR hoạt động đủ nhanh để trả ra lệnh đúng trước khi chu kỳ xung nhịp tiếp theo đánh xuống. Cả 2 hệ thống đều chạy mượt mà ở xung nhịp mục tiêu ($\sim 100 \text{ MHz}$).

IMPLEMENDED DESIGN - Kria Part			
Tcl Console	Messages	Log	Reports
Design Runs	DRC	Methodology	Power
Timing	Utilization		
Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 0.144 ns	Worst Hold Slack (WHS): 0.011 ns	Worst Pulse Width Slack (WPWS): 3.500 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 35186	Total Number of Endpoints: 35122	Total Number of Endpoints: 15307	
All user specified timing constraints are met.			

Hình 22: Timing Summary thiết kế có ISR

IMPLEMENDED DESIGN - Kria Part			
Tcl Console	Messages	Log	Reports
Design Runs	DRC	Methodology	Power
Timing	Utilization		
Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 0.985 ns	Worst Hold Slack (WHS): 0.011 ns	Worst Pulse Width Slack (WPWS): 3.500 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 36893	Total Number of Endpoints: 36893	Total Number of Endpoints: 12307	
All user specified timing constraints are met.			

Hình 23: Timing Summary thiết kế baseline

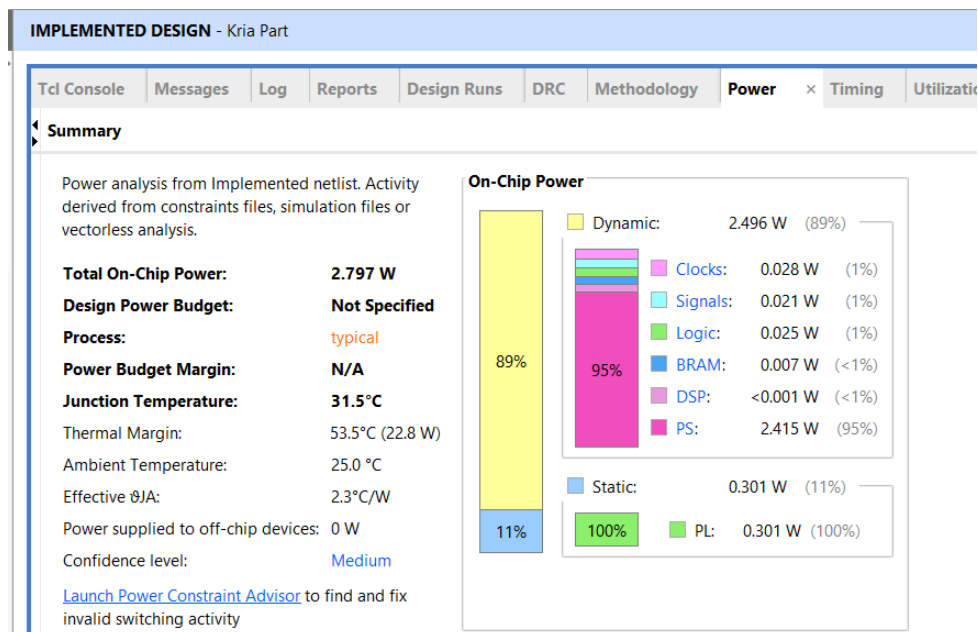
3. Phân tích năng lượng tiêu thụ

Tiêu thụ năng lượng là chỉ số quan trọng thứ 3 sau Diện tích và Timing. Báo cáo ước tính năng lượng của Vivado sau giai đoạn Routing được ghi nhận như sau:

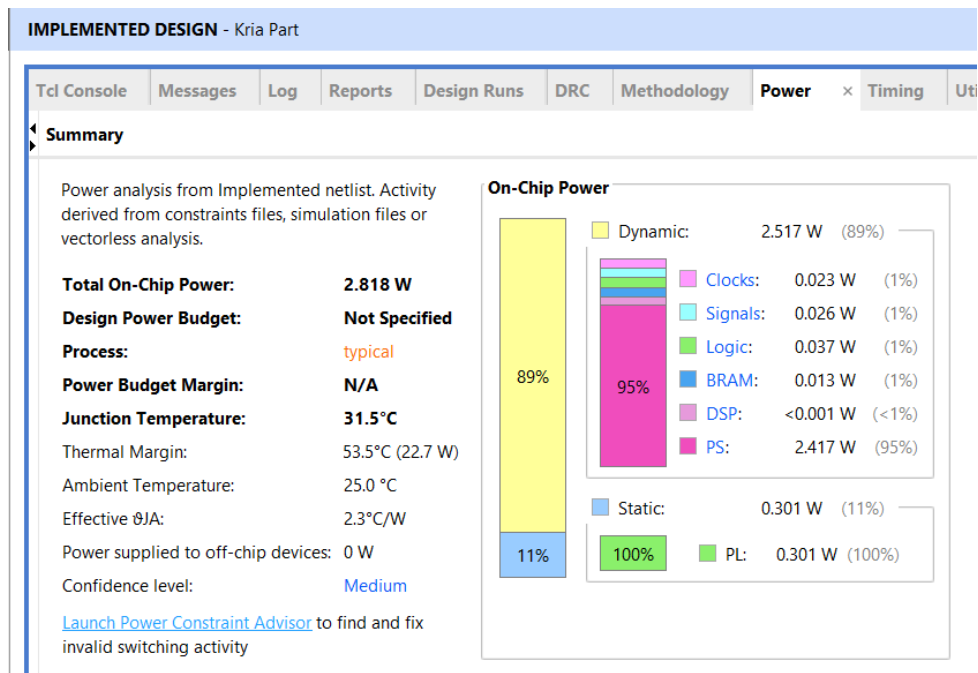
Thành phần năng lượng	Baseline	Ibex-ISR	Mức độ chênh lệch
Total Power	2.818W	2.797W	Giảm 0.021W

Nhận xét:

- Kết quả Vivado ghi nhận mức năng lượng tổng cộng của hệ thống gần như không thay đổi, thậm chí bản có ISR còn tối ưu hơn một chút (2.797W so với 2.818W). Sự biến động rất nhỏ này hoàn toàn là kết quả tự nhiên từ thuật toán tối ưu hóa tự động (Placement & Routing) của phần mềm Vivado.
- Điều này khẳng định mạch giải mã XOR siêu nhẹ không gây ra bất kỳ tác động tiêu cực nào đến bài toán năng lượng và tản nhiệt của bo mạch Kria KV260. Hệ thống bảo mật hoạt động trong suốt mà vẫn cực kỳ tiết kiệm điện.



Hình 24: Power Summary thiết kế có ISR



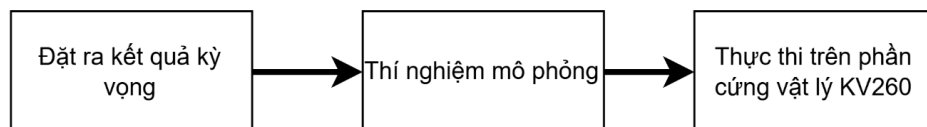
Hình 25: Power Summary thiết kế baseline

Kết quả tổng hợp và triển khai trên công cụ phần mềm chuyên dụng Vivado đã minh chứng bằng số liệu cụ thể cho thấy: Kiến trúc đề xuất không chỉ chính xác về mặt chức năng mà còn đáp ứng xuất sắc các tiêu chuẩn công nghiệp về tài nguyên phần cứng và thời gian. Đây là tiền đề vững chắc để tiến hành nạp bitstream và gỡ lỗi thực tế trên mạch ở phần tiếp theo.

V. KIỂM CHỨNG HỆ THỐNG

Để khẳng định tính đúng đắn của thiết kế và đánh giá mức độ an toàn của cơ chế Instruction Set Randomization (ISR), kịch bản kiểm thử được xây dựng và chia làm 3 nhóm (Test Cases).

Quy trình đánh giá cho mỗi nhóm được tiến hành nghiêm ngặt qua 3 bước:



Hình 26: Quy trình kiểm chứng hệ thống

1. Nhóm 1: Baseline – Không có ISR

Mục tiêu: Xác minh tính đúng đắn của lõi CPU nguyên bản và kịch bản phần mềm trước khi áp dụng cơ chế bảo mật. Tắt cơ chế ISR trên phần cứng (ISR_ENABLE = 0).

- **Kết quả kỳ vọng:** Lõi CPU thực thi chính xác tập lệnh nguyên bản. Kết quả trả về phải vượt qua 3 bài kiểm tra toán học: Phép tính số nguyên cơ bản (ADD/SUB),

Vòng lặp/Hàm đệ quy (Fibonacci), và Mảng dữ liệu (Checksum mảng bằng phép XOR).

Thí nghiệm mô phỏng (Simulation): Nhóm sử dụng công cụ FuseSoC để biên dịch mô hình lõi Ibex thông qua trình giả lập Verilator. Nạp trực tiếp file mã máy dạng text (.vmem) chưa mã hóa vào bộ nhớ ảo. File log Terminal ghi nhận kết quả mô phỏng báo PASS cho toàn bộ các bài toán, chứng minh lõi Ibex gốc hoạt động hoàn hảo.

```
sunlight@sunlight-virtual-machine:~/libex$ ./build/lowrisc_ibex_ibex_simple_system_0/sin-verilator/Vlibex_simple_system -t --meminit=ram,examples/sw/simple_system/lsr_test/lsr_test.elf
Simulation of Ibex
=====
Tracing can be toggled by sending SIGUSR1 to this process:
$ kill -USR1 10632

Simulation running, end by pressing CTRL-C.
Tracing enabled.
Writing simulation traces to sin.fst
TOP:libex_simple_system.u.top.u_ibex_tracer.unnamedblk2.unnamedblk3: Writing execution trace to trace_core_00000000.log
Terminating simulation by software Request.
- src/lowrisc_ibex_sim_shared_0/rtl/sin/simulator_ctrl.sv:93: Verilog $finish
Received $finish() from Verilog, shutting down simulation.

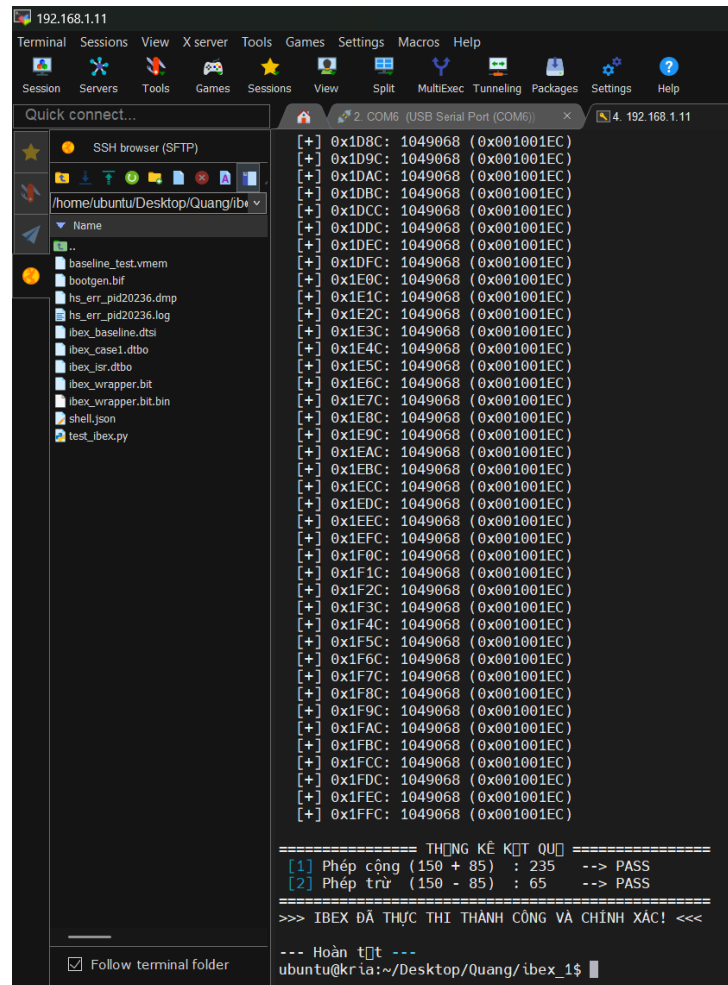
Simulation statistics
=====
Executed cycles: 1038
Wallclock time: 0.021 s
Simulation speed: 87523.8 cycles/s (87.5238 KHz)
Trace file size: 213103 B

You can view the simulation traces by calling
$ gtkwave sin.fst

Performance Counters
=====
Cycles: 1034
Instructions Retired: 1023
sunlight@sunlight-virtual-machine:~/libex$ cat libex_simple_system.log
Hello Ibex with ISR!
Basic math: PASS
Fibonacci: PASS
Array checksum: PASS
Done all tests.
sunlight@sunlight-virtual-machine:~/libex$
```

Hình 27: Kết quả kiểm thử nhóm 1

- **Thực thi trên FPGA KV260:** Trên bo mạch vật lý, sử dụng quy trình phối hợp MobaXterm và Vivado. Lệnh `fpgautil` nạp bitstream, giải phóng AXI Isolation và cấp xung nhịp. Tiếp đó, thông qua giao thức JTAG-to-AXI, kích bản `load_bram.tcl` bơm file mã máy chưa mã hóa vào BRAM. Cuối cùng, gạt VIO nhả Reset cho Ibex chạy và thu được kết quả khớp hoàn toàn với mô phỏng.



Hình 28: Nạp lên kit KV260 lỗi ibex không có ISR

2. Nhóm 2: Có bật ISR, mã hóa đúng

Mục tiêu: Đảm bảo hệ thống hoạt động chính xác, liên mạch từ đầu tới cuối khi áp dụng lớp bảo vệ. Bật `ISR_ENABLE = 1` và áp dụng chung khóa A1B2C3D4 cho cả phần mềm mã hóa và phần cứng giải mã.

- **Kết quả kỳ vọng:** Dữ liệu lệnh lấy ra từ RAM là luồng dữ liệu đã bị mã hóa. Tuy nhiên, khi luồng dữ liệu này đi qua khối ISR Decoder, nó phải được thực hiện phép XOR thứ 2 để khôi phục chính xác 100% về mã lệnh gốc. CPU vượt qua kiểm thử thành công, hệ thống được an toàn mà không làm gián đoạn luồng thực thi hợp lệ.
- **Thí nghiệm mô phỏng:** Sử dụng Script Python `isr_encoder.py` mã hóa file lệnh thành `isr_test_encoded.vmem`. Khởi chạy Verilator với file đã mã hóa này. Quan sát trên dạng sóng, tín hiệu nhận về là mã rác, nhưng lập tức được khối ISR trả ra lệnh RV32I chuẩn trong cùng một chu kỳ xung nhịp. Kết quả mô phỏng xác nhận hệ thống CPU vẫn báo PASS cho các bài toán dù dữ liệu trong RAM là dữ liệu rác đối với người đọc bình thường.

```

192.168.1.11
Terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help

Quick connect...
SSH browser (SFTP)
/home/ubuntu/Desktop/Quang/ibex
Name
...
baseline_test.vmem
ibex_baseline.dtb
ibex_baseline.dtsi
ibex_isr.bit.bin
ibex_wrapper.bit
ibex_wrapper.bit.bin
isr_test_encoded.vmem
isr_test_wrong_key.vmem
shell.json
test_ibex.py

[+] 0x1D8C: 1049068 (0x001001EC)
[+] 0x1D9C: 1049068 (0x001001EC)
[+] 0x1DAC: 1049068 (0x001001EC)
[+] 0x1DBC: 1049068 (0x001001EC)
[+] 0x1DCC: 1049068 (0x001001EC)
[+] 0x1DDC: 1049068 (0x001001EC)
[+] 0x1DEC: 1049068 (0x001001EC)
[+] 0x1DFC: 1049068 (0x001001EC)
[+] 0x1E0C: 1049068 (0x001001EC)
[+] 0x1E1C: 1049068 (0x001001EC)
[+] 0x1E2C: 1049068 (0x001001EC)
[+] 0x1E3C: 1049068 (0x001001EC)
[+] 0x1E4C: 1049068 (0x001001EC)
[+] 0x1E5C: 1049068 (0x001001EC)
[+] 0x1E6C: 1049068 (0x001001EC)
[+] 0x1E7C: 1049068 (0x001001EC)
[+] 0x1E8C: 1049068 (0x001001EC)
[+] 0x1E9C: 1049068 (0x001001EC)
[+] 0x1EAC: 1049068 (0x001001EC)
[+] 0x1EBC: 1049068 (0x001001EC)
[+] 0x1ECC: 1049068 (0x001001EC)
[+] 0x1EDC: 1049068 (0x001001EC)
[+] 0x1EEC: 1049068 (0x001001EC)
[+] 0x1EFC: 1049068 (0x001001EC)
[+] 0x1F0C: 1049068 (0x001001EC)
[+] 0x1F1C: 1049068 (0x001001EC)
[+] 0x1F2C: 1049068 (0x001001EC)
[+] 0x1F3C: 1049068 (0x001001EC)
[+] 0x1F4C: 1049068 (0x001001EC)
[+] 0x1F5C: 1049068 (0x001001EC)
[+] 0x1F6C: 1049068 (0x001001EC)
[+] 0x1F7C: 1049068 (0x001001EC)
[+] 0x1F8C: 1049068 (0x001001EC)
[+] 0x1F9C: 1049068 (0x001001EC)
[+] 0x1FAC: 1049068 (0x001001EC)
[+] 0x1FBC: 1049068 (0x001001EC)
[+] 0x1FCC: 1049068 (0x001001EC)
[+] 0x1FDC: 1049068 (0x001001EC)
[+] 0x1FEC: 1049068 (0x001001EC)
[+] 0x1FFC: 1049068 (0x001001EC)

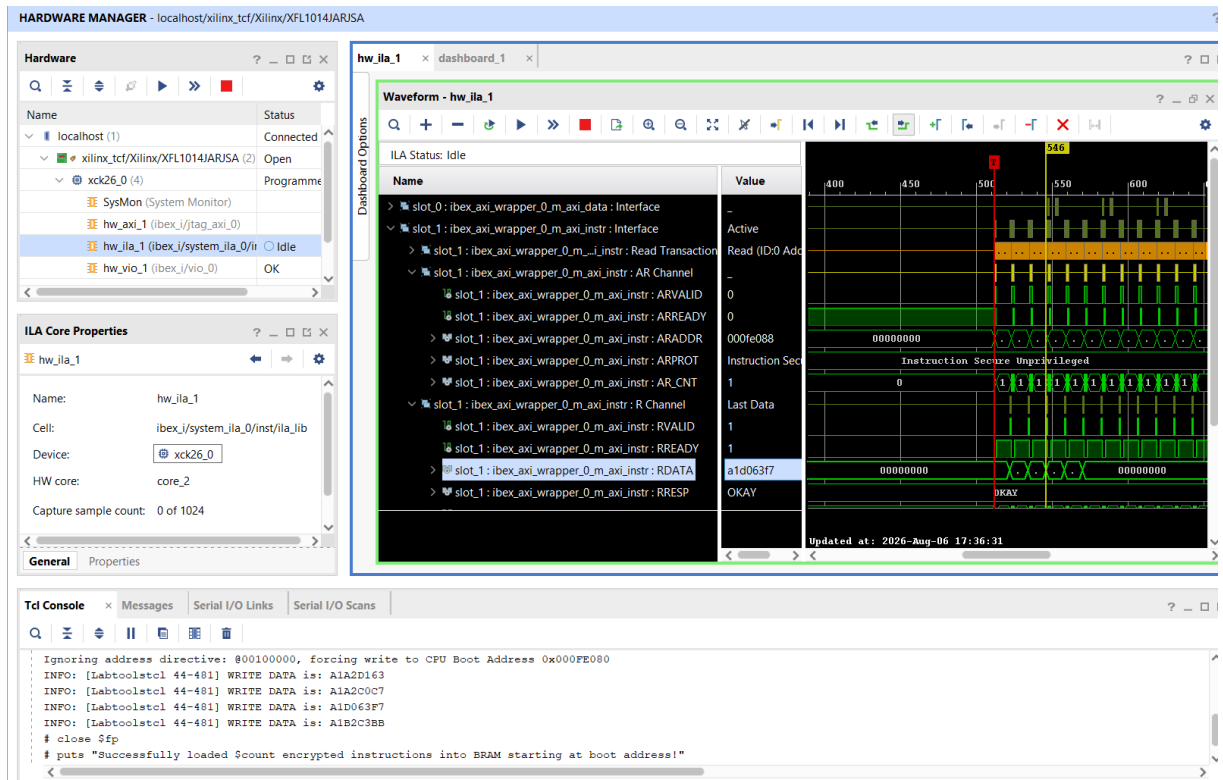
===== TH[NG KẾ K[ T QU[ =====
[1] Phép cộng (150 + 85) : 235 --> PASS
[2] Phép trừ (150 - 85) : 65 --> PASS
=====
>>> IBEX ĐÃ THỰC THI THÀNH CÔNG VÀ CHÍNH XÁC! <<<

--- Hoàn t[ t ---
ubuntu@kria:~/Desktop/Quang/ibex_2$

```

Hình 29: Kết quả kiểm thử nhóm 2

- **Thực thi trên FPGA KV260:** Áp dụng quy trình kiểm thử 4 bước, nạp bitstream qua MobaXterm, đóng băng CPU bằng VIO và bơm dữ liệu mã hóa qua Tcl script. Khi gạt VIO để đánh thức CPU, công cụ ILA lập tức bắt được các tín hiệu sóng sống cho thấy dữ liệu đọc về từ BRAM hoàn toàn là mã hóa, nhưng ngay lập tức đi qua khối ISR và xuất ra lệnh hợp lệ.



Hình 30: Nạp lên kit KV260 lõi ibex có ISR nhóm 2

3. Nhóm 3: Kịch bản tấn công

Mục tiêu: Kiểm chứng cơ chế phòng vệ của phần cứng khi bị kẻ tấn công cố tình chèn mã độc. Kịch bản chia làm 2 trường hợp giả định:

1. Kẻ tấn công không biết hệ thống có ISR → Chèn mã độc dạng Plaintext.
 2. Kẻ tấn công biết hệ thống có ISR nhưng không biết khóa → Chèn mã độc được mã hóa bằng một khóa sai (Ví dụ: 11223344).
- **Kết quả kỳ vọng:** Bởi vì phần cứng đang nắm giữ Khóa đúng (A1B2C3D4), mọi mã lệnh từ kẻ tấn công dù là Plaintext hay mã hóa sai khóa, khi đi qua khối giải mã XOR đều sẽ biến thành một đoạn bit rác không có nghĩa. Tập lệnh RISC-V 32-bit có không gian Opcode rất rộng, nên xác suất cực cao chuỗi rác này sẽ không khớp với bất kỳ lệnh hợp lệ nào. Lõi Ibex phải lập tức kích hoạt cờ Exception và rơi vào trạng thái Illegal Instruction Trap để treo hệ thống, bảo vệ an toàn.
 - **Thí nghiệm mô phỏng:** Tạo ra file `isr_test_wrong_key.vmem` và nạp vào trình mô phỏng Verilator. Kết quả 2 trường hợp đúng như dự đoán:

```
sunlight@sunlight-virtual-machine: ~/libex

292365: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292371: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292377: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292383: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292389: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292395: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292401: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292407: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292413: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292419: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292425: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292431: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292437: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292443: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292449: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292455: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292461: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292467: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292473: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292479: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292485: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292491: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292497: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292503: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292509: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292515: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292521: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292527: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292533: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292539: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292545: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292551: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292557: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292563: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292569: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292575: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
292581: Illegal instruction (hart 0) at PC 0x00100000: 0x9752c3bb
^CReceived stop request, shutting down simulation.

Simulation statistics
=====
Executed cycles: 146292
Wallclock time: 2.726 s
Simulation speed: 53665.4 cycles/s (53.6654 kHz)
Trace file size: 3472589 B

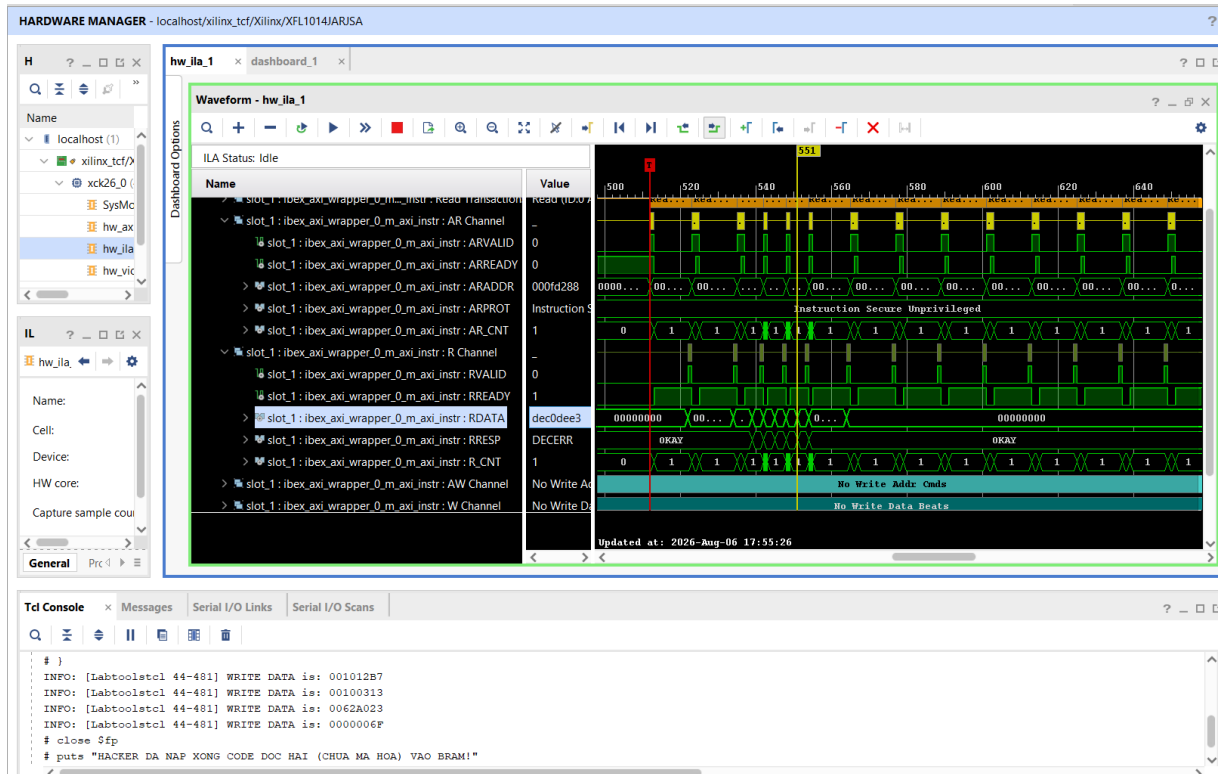
You can view the simulation traces by calling
$ gtkwave sim.fst

Performance Counters
=====
Cycles: 146287
Instructions Retired: 0
sunlight@sunlight-virtual-machine:~/libex$
```

Input to this VM: move the mouse pointer inside or press Ctrl+C

```
290199: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290205: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290211: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290217: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290223: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290229: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290235: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290241: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290247: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290253: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290259: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290265: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290271: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290277: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290283: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290289: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290295: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290301: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290307: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290313: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290319: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290325: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290331: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290337: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290343: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290349: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290355: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290361: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
290367: Illegal instruction (hart 0) at PC 0x00100000: 0x8670f0ff
```

- **Thực thi trên FPGA KV260:** Trên kit thực tế, nạp file mã độc vào BRAM qua JTAG và gài bẫy ILA tại tín hiệu trả về lỗi (DECERR/SLVERR) của bus AXI. Ngay khi nhả VIO Reset, ILA lập tức chớp được khoảnh khắc Ibex đọc lệnh đầu tiên, giải mã sai thành Opcode rác và rơi thẳng vào trạng thái Halt. Lỗi CPU ngừng phát sinh giao dịch, hệ thống hoàn toàn câm lặng, chứng tỏ mã độc đã bị cô lập và tiêu diệt ngay từ tầng Instruction Fetch.



Hình 32: Nạp lên kit KV260 lỗi ibex có ISR nhóm 3

VI. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết luận

Đồ án đã nghiên cứu và hiện thực thành công kỹ thuật Instruction Set Randomization (ISR) tích hợp trực tiếp vào đường ống lệnh (pipeline) của vi xử lý RISC-V Ibex mã nguồn mở. Những kết quả nổi bật đạt được trong khuôn khổ đồ án bao gồm:

1. **Thiết kế phần cứng Zero-cycle Latency:** Việc sử dụng thuật toán mã hóa XOR tổ hợp giúp quá trình giải mã tại tầng Instruction Fetch diễn ra tức thời. Hệ thống không sinh ra bất kỳ chu kỳ trễ nào, qua đó bảo toàn trọn vẹn cơ chế xử lý luồng và kiến trúc đường ống 2 tầng nguyên bản của lõi Ibex.
2. **Kết quả tổng hợp trên phần mềm Xilinx Vivado chứng minh module ISR chỉ tiêu tốn thêm khoảng 4.3% lượng LUTs** và gần như không làm thay đổi lượng điện năng tiêu thụ tổng thể. Khả năng đáp ứng Timing ở tần số $\sim 100\text{MHz}$ khẳng định tính khả thi cao khi đưa thiết kế này vào các thiết bị IoT/Nhúng hạn chế tài nguyên.

3. **Hoàn thiện công cụ phần mềm:** Xây dựng thành công công cụ mã hóa Python có khả năng phân tách thông minh giữa vùng nhớ lệnh và vùng nhớ dữ liệu, đảm bảo mã máy được mã hóa toàn vẹn mà không làm hỏng tính logic của dữ liệu hằng số (.rodata/.data).
4. **Kiểm chứng vật lý toàn diện trên Kria KV260:** Đồ án không dừng lại ở mức mô phỏng phần mềm mà đã triển khai thành công một quy trình gỡ lỗi phần cứng. Sự kết hợp giữa Zynq PS (quản lý qua MobaXterm), công tắc ảo VIO, kênh nạp dữ liệu JTAG-to-AXI và máy phân tích logic nhúng ILA đã cung cấp những minh chứng trực quan về sự hoạt động của ISR ở mức độ tín hiệu điện áp vật lý.
5. **Ngăn chặn thành công tấn công mã độc:** Qua các kịch bản kiểm, đồ án khẳng định lõi CPU có khả năng tự động phát hiện và vô hiệu hóa các nỗ lực chen mã độc hoặc nạp sai khóa bảo mật. Mọi lệnh sai lệch đều lập tức đẩy CPU rơi vào trạng thái bất ngoại lệ và đóng băng hệ thống, chặn đứng quyền điều khiển của kẻ tấn công.

Tóm lại, đề tài đã giải quyết trọn vẹn mục tiêu ban đầu đề ra: Xây dựng một khiên bảo vệ chủ động ngay từ cấp độ kiến trúc phần cứng, ngăn chặn các dạng tấn công chen mã độc.

2. Hướng phát triển

Mặc dù hệ thống hiện tại đã hoạt động ổn định và chứng minh được tính hiệu quả của kỹ thuật ISR, vẫn còn nhiều không gian để cải tiến và nâng cấp đồ án lên các tiêu chuẩn bảo mật cao hơn trong tương lai:

1. **Cập nhật Khóa động và Khóa dùng một lần:** Phiên bản hiện tại đang sử dụng một khóa tĩnh 32-bit. Để chống lại các cuộc tấn công dò khóa hoặc phân tích kênh kề, hệ thống có thể được nâng cấp bằng cách tích hợp một Bộ tạo số ngẫu nhiên thực (TRNG) hoặc LFSR trên FPGA. Từ đó, mỗi lần vi xử lý khởi động hoặc thay đổi tiến trình, hệ thống sẽ tự động sinh ra một khóa mã hóa hoàn toàn mới.
2. **Áp dụng thuật toán Mã hóa Khối nhẹ:** Thuật toán XOR tuy nhanh nhưng có độ bền mật mã học chưa cao nếu bị kẻ gian thu thập đủ cặp Bản rõ - Bản mã. Hướng phát triển tiếp theo là thay thế cổng XOR bằng các thuật toán mã hóa khối chuẩn công nghiệp như PRINCE hay PRESENT. Để giải quyết rào cản về Timing của các thuật toán này, có thể áp dụng kỹ thuật Pipelining the Cipher để duy trì thông lượng lệnh ở mức tối đa.
3. **Mở rộng bảo vệ vùng nhớ Dữ liệu:** Đồ án hiện tại mới chỉ tập trung cô lập vùng nhớ chứa Lệnh. Việc mở rộng thuật toán để mã hóa cả vùng nhớ Dữ liệu sẽ giúp hệ thống chống lại được toàn diện các loại hình tấn công tinh vi hơn nhắm vào việc làm rò rỉ bộ nhớ hoặc ghi đè con trỏ hàm.
4. **Hỗ trợ đa nhiệm cấp độ Hệ điều hành:** Phát triển cơ chế phần cứng (thông qua các thanh ghi Control and Status Registers - CSRs) để hệ điều hành chạy trên Ibex có thể tự động cấp phát và quản lý các khóa ISR khác nhau cho từng tiến trình riêng biệt. Điều này đảm bảo sự cách ly tài nguyên: Nếu một tiến trình bị xâm nhập, các tiến trình khác trên cùng một lõi vật lý vẫn được bảo vệ tuyệt đối.

TÀI LIỆU THAM KHẢO

- [1] G. S. Kc, A. D. Keromytis, and V. Prevelakis, "Countering code-injection attacks with instruction-set randomization,"in *Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS)*, Washington D.C., USA, 2003, pp. 272–280. DOI: 10.1145/948109.948146.
- [2] E. G. Barrantes, D. H. Ackley, S. Forrest, T. S. Palmer, D. Stefanović, and D. D. Zovi, "Randomized instruction set emulation (RISE) to thwart code injection attacks,"in *Proceedings of the 10th ACM Conference on Computer and Communications Security (CCS)*, Washington D.C., USA, 2003, pp. 281–289. DOI: 10.1145/948109.948147.
- [3] H. Shacham, "The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86),"in *Proceedings of the 14th ACM Conference on Computer and Communications Security (CCS)*, Alexandria, VA, USA, 2007, pp. 552–561. DOI: 10.1145/1315245.1315313.
- [4] S. Sinha, H. K. Kapoor, and S. Biswas, "Hardware-assisted Instruction Set Randomization for Secure Embedded Processors,"in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 8, pp. 1450–1461, Aug. 2021. DOI: 10.1109/TVLSI.2021.3087220.
- [5] P. D. Schiavone, D. Rossi, A. Pullini, A. Di Mauro, F. Conti, and L. Benini, "Slow and steady wins the race? A comparison of ultra-low-power RISC-V cores for Internet-of-Things applications,"in *27th International Symposium on Power and Timing Modeling, Optimization and Simulation (PATMOS)*, Thessaloniki, Greece, 2017, pp. 1–8. DOI: 10.1109/PATMOS.2017.8106976.
- [6] A. Waterman and K. Asanović (Eds.), *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2*, RISC-V Foundation, May 2017. [Online]. Available: <https://riscv.org/specifications/>
- [7] A. Waterman and K. Asanović (Eds.), *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10*, RISC-V Foundation, May 2017. [Online]. Available: <https://riscv.org/specifications/privileged-isa/>
- [8] lowRISC Contributors, *Ibex RISC-V Core Reference Guide*, lowRISC CIC, Cambridge, UK, 2023. [Online]. Available: <https://ibex-core.readthedocs.io/>
- [9] OpenTitan Project, *OpenTitan: Open Source Silicon Root of Trust*, lowRISC & Google, 2022. [Online]. Available: <https://opentitan.org/>
- [10] ARM Ltd., *AMBA AXI and ACE Protocol Specification: AXI3, AXI4, and AXI4-Lite, ACE and ACE-Lite (ARM IHI 0022E)*, ARM Limited, Cambridge, UK, 2011.
- [11] AMD Xilinx Inc., *Zynq UltraScale+ Device Technical Reference Manual (UG1085)*, v2.3, AMD Xilinx, San Jose, CA, USA, Jun. 2022.
- [12] AMD Xilinx Inc., *Kria KV260 Vision AI Starter Kit User Guide (UG1089)*, v1.2, AMD Xilinx, San Jose, CA, USA, May 2022.
- [13] AMD Xilinx Inc., *Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator (UG994)*, v2023.1, AMD Xilinx, San Jose, CA, USA, 2023.

- [14] J. Borghoff, A. Canteaut, T. Güneysu, E. B. Kavun, M. Knezevic, L. R. Knudsen, G. Leander, V. Nikov, C. Paar, C. Rechberger, P. Rombouts, S. S. Thomsen, and T. Yalçin, "PRINCE – A Low-latency Block Cipher for Pervasive Computing Applications,"in *Advances in Cryptology – ASIACRYPT 2012*, Lecture Notes in Computer Science, vol. 7658, Springer, Berlin, Heidelberg, 2012, pp. 208–225. DOI: 10.1007/978-3-642-34961-4_14.
- [15] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Viskelson, "PRESENT: An Ultra-Lightweight Block Cipher,"in *Cryptographic Hardware and Embedded Systems (CHES 2007)*, Lecture Notes in Computer Science, vol. 4727, Springer, Berlin, Heidelberg, 2007, pp. 450–466. DOI: 10.1007/978-3-540-74735-2_31.
- [16] W. Snyder et al., *Verilator: Open-Source System Verilog Simulator and C++ Testbench Framework*, Veripool, 2023. [Online]. Available: <https://www.veripool.org/verilator/>
- [17] O. Kindgren, *FuseSoC: An Award-winning Package Manager and Build Tool for HDL Code*, 2022. [Online]. Available: <https://github.com/olofk/fusesoc>
- [18] IC-design-lab-HCMUT, *Instruction-Set-Randomization*. [Private]. Available: <https://github.com/IC-design-lab-HCMUT/Instruction-Set-Randomization>